MODULE 8

Dify 應用開發：從 Chatbot 到 Agent

M8 — Building LLM Applications with Dify

把模型變成能用的系統。三週走完 Chatbot → RAG 知識庫 → Agent，
並以台灣長照 3.0 為實例，最後交付你自己的期末專題。

資料科學　2027 Spring　·　第 13–15 週　·　Phase C　|　謝楠楨

本講義大綱

01 Dify 是什麼、不是什麼 • LLM 應用的組裝平台

02 四種應用型態的流程 • 各自的節點結構與取捨

03 環境建置與成本 • 雲端 vs 本地、 Token 怎麼算

04 System Prompt 四段式 • 角色・步驟・限制・格式

05 抑制幻覺的手法 • 從提示詞層面下手

06 RAG 四步流程 • 切塊・嵌入・召回・生成

07 切塊與召回設定 • 實務上真正要調的參數

08 Agentic RAG • 2026 的新做法

09 RAG 五種失效模式 • 以及各自的處方

10 ★ 可操作實例 • 從零建一個長照給付試算 Chatflow

11 Workflow / Chatflow / Agent • 三種模式的選用時機

12 Agent 思考迴圈與工具 • Thought → Call Tool → Output

13 MCP 與外部整合 • 標準化的工具接法

14 異常處理與 Fallback • 流程不會永遠順利

15 Human-in-the-Loop • 醫療場域的權責分工

16 期末專題規格 • 構想書與實作驗收

17 工具全景與選型 • n8n 等其他工具的定位

從 M7 到 M8：你已經有的基礎

M7 學過的	在 M8 怎麼用
Transformer 與 Attention 的直覺	理解為什麼上下文長度有限、為什麼 Token 要計費
預訓練／微調／推論的差別	Dify 只做「推論 + 提示」，不做訓練——這決定了它能與不能做什麼
Token 與上下文長度	估算 System Prompt ＋知識庫段落會不會超出上限
Hugging Face 與 Ollama	選擇模型供應商；本地部署時接 Ollama
提示詞的基本概念	本模組把它系統化成「四段式 System Prompt」

> M7（W12）是本模組的前置。若對 Token、上下文、微調與推論的差別還不清楚，先回頭複習那一份。

三週的教學安排

週次	主題	課堂實作	課後
W13	平台入門、 Chatbot 與提示詞工程 PART 1–4	教學 01 vs 教學 02 對照； AI Chatbot (healthcare)	Slogan Generator 1/2 Company Research ； ★ 構想書發布
W14	RAG 與知識庫 PART 5–6	長照 3.0 知識庫建置； 召回測試與參數調整	CMS Workflow ； 三個模型版本比較
W15	Workflow 、 Agent 與工具調用 PART 7–9	Workflow vs Agent 對照； 教學 03 健康主題查證 Agent	Movie Researcher 兩版； ★ 構想書繳交、專題實作衝刺

每週正式授課 2 小時。 W15 課後即進入專題衝刺——構想書 W15 繳交、 Demo W16 發表，時間很緊，請提早動工。

PART 01

Dify 是什麼

What Dify Is — and Isn't

它不是另一個 ChatGPT，也不是一個模型。
它是把 LLM、知識庫、外部工具與流程邏輯「組裝」起來的地方。

Dify 是什麼／不是什麼

Dify 是

- LLM 應用的組裝平台——你在上面接線，不是訓練模型
- 可視化流程編輯器，概念上很接近 AI Studio 的流程圖
- 內建 RAG 知識庫：上傳文件即可做檢索增強
- 支援多家模型供應商，可隨時切換底層 LLM
- 應用可一鍵發佈為公開網頁或嵌入式元件
- 有完整的 Tracing 日誌，看得到每一步在想什麼

Dify 不是

- 不是模型訓練工具——它不做預訓練，只做應用層
- 不會讓爛 Prompt 變好——垃圾進、垃圾出的規則依然適用
- 不保證正確性—— LLM 會幻覺， Dify 只是幫你組裝
- 不是免費的——每次呼叫都在燒 Token ，成本要自己控制
- 不能取代領域知識——知識庫的品質決定答案的品質
- 不適合毫秒級延遲或嚴格確定性輸出的場景

定位比較： AI Studio （ M1–M5 ）處理「有標準答案的預測問題」， Dify 處理「需要語言理解與知識檢索的決策支援問題」。兩者互補，不是替代。

與前面模組的對照

面向	AI Studio （M1–M5）	Dify （M8）
處理的問題	有明確目標變數的預測	需要語言理解與知識檢索的決策支援
輸入	結構化表格	自然語言 ＋ 文件 ＋ 即時資料
輸出	數值或類別（一個分數）	文字、建議、行動（一段回答）
怎麼驗證	與正確答案比對（Accuracy、 AUC）	人工審閱＋來源可追溯性
失敗的樣子	預測不準（看得出來）	講得頭頭是道但是錯的（看不出來）
對應的把關	交叉驗證、指標選擇	Prompt 約束、 RAG 來源標註、人在迴路

最後兩列是本模組的核心： LLM 的錯誤與傳統模型的錯誤，長得完全不一樣，所以把關方式也完全不同。

四種應用型態



★ 選型的第一個問題不是「哪個功能多」，而是「這個任務容不容許路徑不固定」。

能用 Workflow 解決的就不要用 Agent —— 自主性是能力，也是不可預測性的來源。

Chatbot

一問一答。行為完全由 System Prompt 決定，沒有流程可言

Workflow

路徑固定，一步接一步。可預測、好除錯，適合批次處理

Chatflow

有對話又有條件分支。分支數有限、可窮舉

Agent

★ LLM 自己決定下一步。最強也最不可預測——專題一要求你論證為何非它不可

圖：課程自製 ★原生圖形，可直接在 PowerPoint 編輯

四種型態怎麼選

	Chatbot	Workflow	Chatflow	Agent
路徑	無流程	固定	有限分支	LLM 決定
對話	單輪或多輪	無	多輪	多輪＋可追問
可預測性	中	★ 最高	高	最低
除錯難度	低	低	中	高（需讀 Tracing）
適合	衛教問答 資訊查詢	批次文件處理 報告生成	分流式評估 有限狀態流程	任務模糊 需組合多工具
本課範例	AI Chatbot (healthcare)	Marketing Plan Generator	台灣長照 3.0 Chatflow	Component-Agent Movie Researcher

> 專業判斷：能用 Workflow 解決的就不要用 Agent。自主性是能力，也是不可預測性的來源——醫療場域要為此付代價。

Dify 與其他方案的定位

方案	形態	適合	限制
Dify	低程式碼平台，可自架	★ 本課程；快速建置可分享的應用；需要 RAG 與流程編排	複雜自訂邏輯仍受平台限制
直接呼叫 API	自己寫程式（Python 等）	需要完全客製化；要整合進既有系統	需要程式能力；所有基礎設施自建
其他低程式碼平台	Flowise、n8n 等	各有偏重（n8n 偏自動化流程）	生態與文件成熟度不一
商用套裝	各廠商的院內 AI 方案	有廠商支援與責任歸屬	彈性低；成本高；需走採購流程

本課程選 Dify 的理由：不需程式基礎、涵蓋 Chatbot 到 Agent 的完整光譜、可本地部署（資料不出機器）、且能一鍵發佈供 Demo。

PART 02

四種應用型態的流程結構

Chatbot / Workflow / Chatflow / Agent

不是四個功能選單，而是四種「流程可控程度」（文字生成器歸入 Chatbot ）。
選錯型態，後面的 Prompt 再怎麼調都補不回來。

先建立一個判斷軸：流程由誰決定?

① 沒有流程　Chatbot

- 整個應用＝一段 System Prompt， LLM 收到問題就直接回答
- 你能控制的只有「怎麼問」，控制不了「它怎麼做」
- → 適合單輪問答；不適合需要保證步驟的場景

② 流程由你決定　Workflow / Chatflow

- 你在畫布上把節點連好，執行時就照這條線走
- 每次 Tracing 長得幾乎一樣 → 可重現、可稽核
- → 醫療、法規、給付試算類應用的主力形態

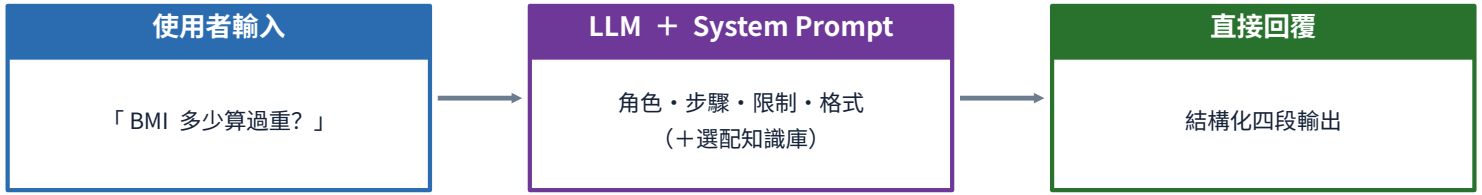
③ 流程由 LLM 決定　Agent

- 你只給目標與工具，路徑由 LLM 每一輪自己決定
- 同樣輸入，兩次執行可能走不同路 → 不可重現
- → 任務模糊、需拆解、需綜合多來源時才值得

★ 選型的第一個問題不是「哪個功能多」，而是「這個任務容不容許路徑不固定」。

模式一　Chatbot：一問一答，沒有流程

整個應用就是一段 System Prompt：你能控制「怎麼問」，控制不了「它怎麼做」



三個方塊之間沒有可編輯的節點——這條線是固定的，你動不了它

你唯一能控制的東西： System Prompt

① 角色	你是長照衛教助理，服務對象是家庭照顧者
② 步驟	先確認對象與情境，再給建議，最後提醒就醫時機
③ 限制	不提供藥物劑量、不做診斷、不取代專業評估
④ 格式	固定四段：重點 → 說明 → 注意事項 → 免責聲明

你控制不了的事

- ✕ 保證它「先查資料再回答」
- ✕ 在中途插入計算或驗證
- ✕ 依條件走不同的分支
- ✕ 失敗時自動重試一次
- ✕ 每次都走完全一樣的路

★ 沒有流程可編輯：畫布上沒有節點。想改行為，只能改 Prompt 文字。

適合：衛教問答、資訊查詢這類「問了就答」的單輪或多輪場景。

課程範例： AI Chatbot (healthcare).yml 、教學 01 ／教學 02 衛教問答對照組。

結構

使用者輸入 → LLM （＋ System Prompt ）→ 直接回覆。畫布上沒有節點可以編輯。

能控制什麼

只有 Prompt 。想改行為就只能改 Prompt 文字。

控制不了什麼

無法保證它「先查資料再回答」，也無法在中途插入計算或驗證。

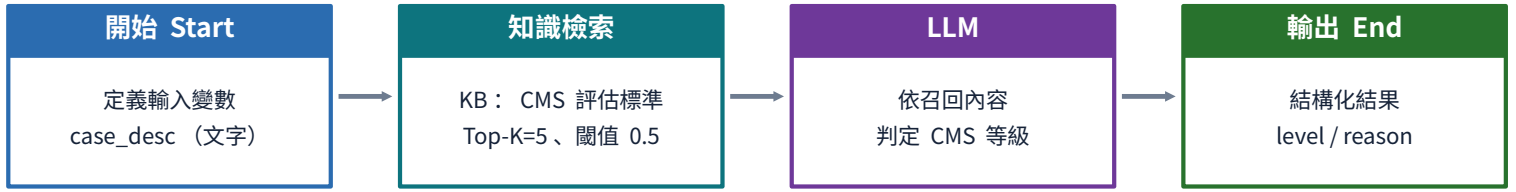
課程範例

AI Chatbot (healthcare).yml 、教學 01 ／教學 02 衛教問答對照組

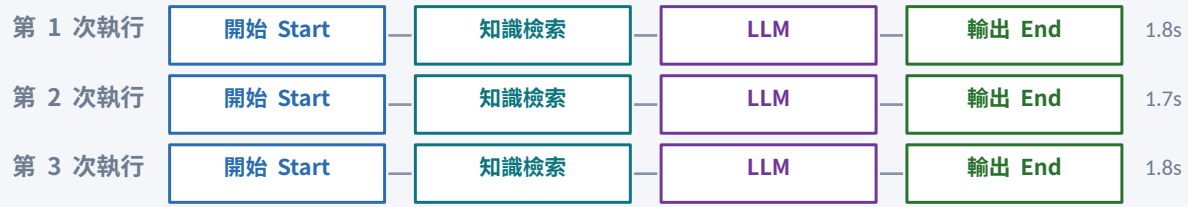
mode = chat / agent-chat （無 workflow 區塊）　★ 原生圖形，可直接在 PowerPoint 編輯

模式二　Workflow： 路徑固定的自動化流程

4 個節點、一條線：這就是 CMS Workflow - case assessment-01.yml 的全部



同樣的輸入，跑三次的 Tracing 長得一模一樣



→ 可重現、可稽核：批次跑 50 份也放心，任何一次出錯都能逐次比對找出差異。

★ 不保留對話歷史： Workflow 是「跑一次給一個結果」，不是聊天。

同樣輸入永遠同樣路徑， Tracing 可以逐次比對——這是可稽核性的來源。

適合：批次跑 50 份個案摘要、把非結構化文字轉成表格、定期生成報表。

真實範例： CMS Workflow - case assessment-01.yml （4 節點： start → 知識檢索 → LLM → end ）。

結構

開始 → 節點 → 節點 → 輸出。沒有對話介面，是「跑一次給一個結果」。

關鍵差別

不保留對話歷史。同樣輸入永遠同樣路徑，適合批次處理。

真實範例

CMS Workflow - case assessment-01.yml：
只有 4 個節點
start → 知識檢索 → LLM → end

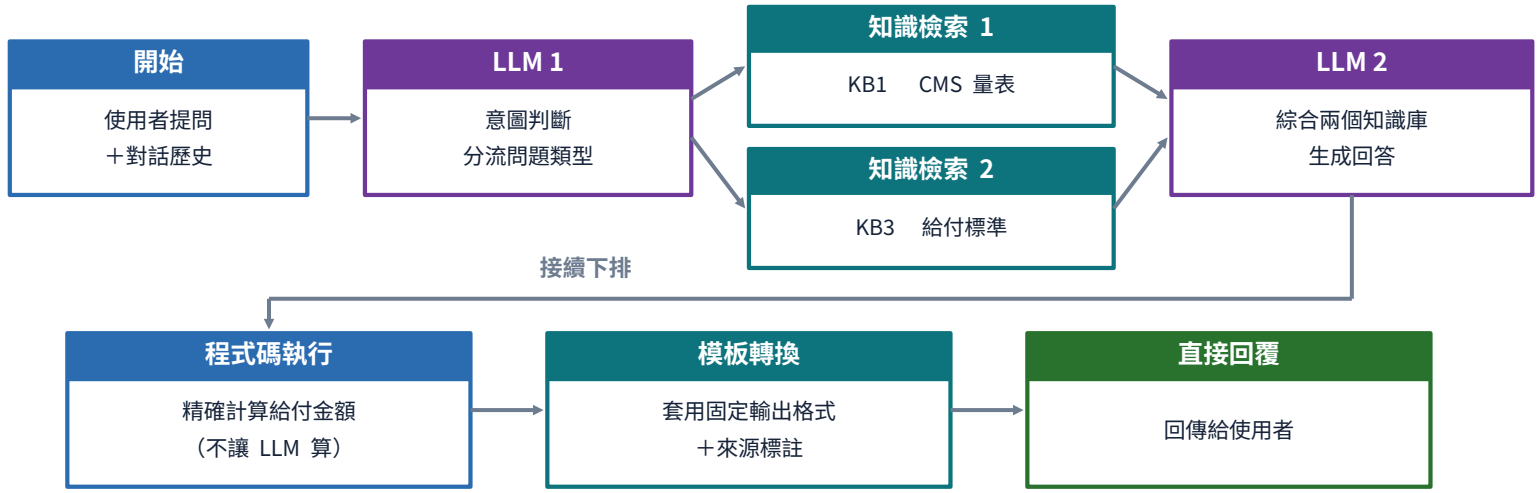
什麼時候用

批次跑 50 份個案摘要、把非結構化文字轉成表格、定期生成報表。

mode = workflow　★ 原生圖形，可直接在 PowerPoint 編輯

模式三　Chatflow： 對話 + 條件分支

先判斷意圖再決定查哪個庫；金額交給程式算，最後套固定格式輸出



為什麼要兩個知識庫

CMS 量表與給付標準是兩份性質不同的文件。
混在同一個庫，量表題目會干擾金額段落的召回。

★ 為什麼要程式碼節點

給付金額用程式查表算，不讓 LLM 生成。
這是本模組最重要的一條規則——數字不能靠猜。

★ 有對話、有分支，但分支數量有限且可窮舉——這是它跟 Agent 最關鍵的差別。

LLM 1 先判斷問題類型，再決定查哪個知識庫；每一條路都是你事先畫好的。

真實範例：台灣長照 3.0 Chatflow （15 節點，含 2 知識檢索、 2 LLM 、 1 程式碼、 1 模板轉換）。

結構

有對話歷史，也有畫布。可以先用一個 LLM 判斷意圖，再決定走哪條線。

為什麼要兩個知識庫

CMS 量表與給付標準是兩份性質不同的文件。

混在同一個知識庫會互相干擾召回。

為什麼要程式碼節點

★ 給付金額用程式算，不讓 LLM 生成。
這是本模組最重要的一條規則。

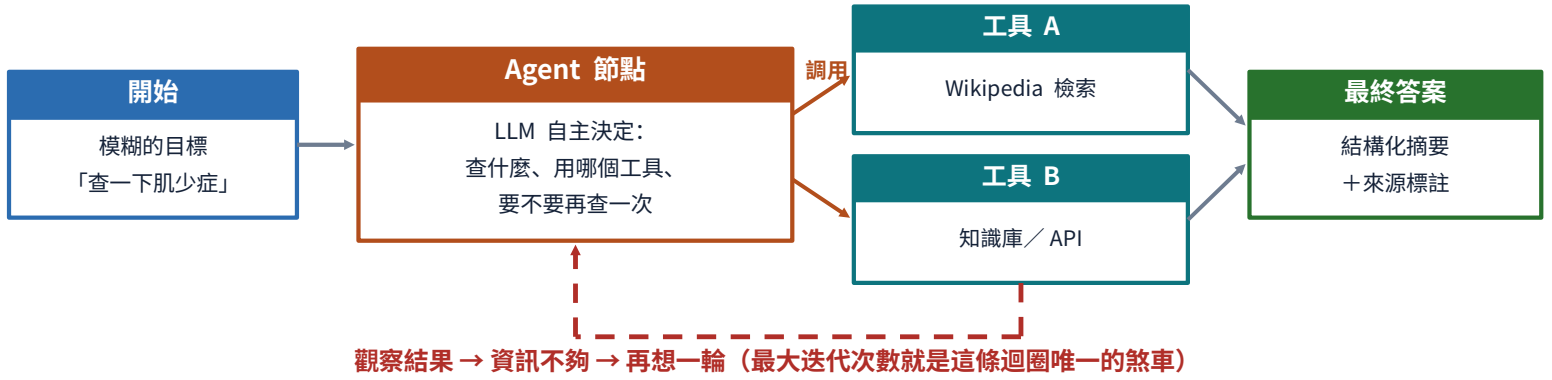
真實範例

台灣長照 3.0 Chatflow： 15 節點
（2 知識檢索、 2 LLM 、 1 程式碼、 1 模板轉換）

mode = advanced-chat　★ 原生圖形，可直接在 PowerPoint 編輯

模式四　Agent： LLM 自己決定下一步

你交出去的不是流程，而是一組工具與一句目標——路怎麼走由它自己決定



一次真實執行的迭代紀錄

第 1 輪	Thought： 先弄清楚定義	→ 工具 A： Wikipedia 「肌少症」條目
第 2 輪	Thought： 缺台灣本地盛行率	→ 工具 B： 知識庫「國健署調查報告」
第 3 輪	Thought： 證據夠了，可以收手	→ 產出最終答案並標註兩個來源

★ 下一次執行同樣的問題，可能只跑 2 輪，也可能一路跑到上限——你事先不知道。

★ 路徑不固定：同樣的輸入，兩次執行可能調用不同工具、迭代不同輪數。

你不能事先畫出「它會怎麼走」——你只能規範它「可以用什麼工具、不能做什麼」。

代價：不可重現、成本不可預估、除錯困難（每次 Tracing 都不一樣）。

課程範例： Component-Agent-01.yml （3 個工具）、教學 03 健康主題查證 Agent.yml。

結構

你不畫流程，你給工具。 LLM 每一輪自己決定要不要再查、查什麼。

紅色虛線是重點

觀察結果 → 資訊不夠 → 再想一輪。這個迴圈的次數由 LLM 決定，
你只能設「最大迭代次數」當煞車。

代價

不可重現、成本不可預估、除錯困難（每次 Tracing 都不一樣）。

課程範例

Component-Agent-01.yml （3 個工具）、教學 03 健康主題查證 Agent

mode = agent-chat 或 workflow 內含 Agent 節點　★ 原生圖形，可直接在 PowerPoint 編輯

同一個需求，四種做法的取捨

① Chatbot 做法

「長照給付怎麼算？」
→ 依 Prompt 寫死的規則

使用者問 → LLM + Prompt → 回覆

最快，但規則一改就要改 Prompt，而且無法處理個案差異

② Chatbot ＋知識庫

「長照給付怎麼算？」
→ 召回原文後再回答

使用者問 → 知識檢索 → LLM → 回覆

法規更新只要換文件；但仍然不能做個案計算——LLM 算數學不可靠

③ Workflow ／ Chatflow

輸入個案描述 → 判等級
→ 程式碼算金額 → 輸出

輸入個案 → 知識檢索 → LLM 判等級 → 程式碼算金額 → 模板輸出

★ 數字用程式算，不讓 LLM 生成。多數醫療應用的正解

④ Agent

「這位個案能申請什麼？」
→ 自行決定查什麼、算什麼

模糊需求 → Agent 思考 → 查表工具 → 計算工具 → 答案

最彈性，但不可預測。需要多來源綜合判斷時才值得

★ 從 ① 到 ④，可靠度與可維護性一路改變——選型就是在這條線上挑一個點。

能用 ③ 解決的就不要用 ④：自主性是能力，也是不可預測性的來源。

這張圖是本模組的選型總結

同樣一句「長照給付試算」，四種做法都做得出來，

但可靠度與可維護性差很多。

① 最快但最脆

規則寫死在 Prompt 裡。法規一改，Prompt 全要改。

② 解決了「資料更新」

但仍然不能做個案計算 —— LLM 算數學不可靠。

③ ★ 多數醫療應用的正解

LLM 只做「判斷分類」，數字交給程式碼節點。

④ 保留給真正模糊的任務

需求本身說不清楚要查什麼時，Agent 才有價值。

★ 原生圖形，可直接在 PowerPoint 編輯

四種模式的規格對照

	Chatbot	Workflow	Chatflow	Agent
對話歷史	✔	✘	✔	✔
可編輯畫布	✘	✔	✔	部分
條件分支	✘	✔	✔	LLM 自訂
路徑可預測	✔	✔✔	✔	✘
可插入程式碼計算	✘	✔	✔	經工具
每次執行成本	固定・低	固定	固定	不固定・高
除錯難度	低	低	中	高
醫療場域適用度	限單輪衛教	★ 批次	★★ 主力	需嚴格護欄

> 選型口訣：能用 Workflow 就別用 Agent；需要對話就升 Chatflow；只有「連你自己都說不清要查什麼」才用 Agent。

隨堂練習：這個任務該用哪一種應用型態？

1 把 200 份出院病摘轉成結構化表格

Workflow

不需要對話、路徑固定、要能批次跑。用 Chatflow 反而多此一舉

2 衛教問答：「BMI 多少算過重？」

Chatbot

問了就答的單輪問答，一段四段式 System Prompt 就夠

3 家屬多輪描述個案 → 判定等級 → 查表算給付

Chatflow

有對話、有分支、要插入程式碼計算——長照 3.0 Chatflow 就是這一型

4 「這個主題的最新證據有哪些？」要自己決定查哪些來源、查幾次

Agent

需求說不清楚要查什麼、要組合多個外部來源，且允許路徑不可重現

5 每週一自動生成上週個案統計報告

Workflow

固定步驟、不需要對話

6 依問題類型，分流到 CMS 量表或給付標準兩個知識庫

Chatflow

分支數量有限、可窮舉——畫得出流程圖，就不需要 Agent

判準：畫得出流程圖 → Chatflow ／ Workflow；需要對話 → Chatbot 或 Chatflow；連要查什麼都說不清楚、允許路徑不可重現 → 才是 Agent。專題一要求你論證「為何非用這個模式不可」。

PART 03

環境建置與成本

Setup & Cost

課堂用雲端版最快。

但你的專題若處理健康資料，本地部署不是選項而是必要條件。

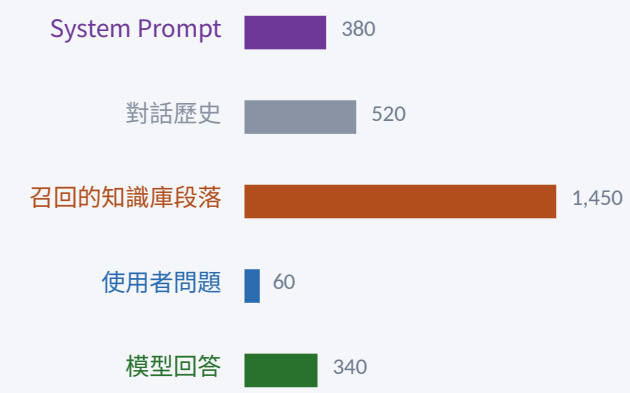
雲端版 vs 本地部署

	Dify Cloud	本地部署（Docker）
建置成本	註冊即用，五分鐘上手	需安裝 Docker，約 30–60 分鐘
資料落地	對話與知識庫存放在雲端	全部留在自己的機器上
模型選擇	接雲端 API	可接雲端，也可搭配 Ollama 完全離線
成本	免費額度有限，超出需付費	硬體自負；用本地模型則無 API 費用
適用	課堂練習、快速原型、專題 Demo	涉及敏感資料、機構有資安要求時
課程建議	W13–W15 課堂練習用這個	專題若處理醫療資料，改用本地部署

> 完整安裝步驟見教材 Dify local install.pdf。搭配 Ollama（M7 已安裝）可做到資料完全不出機器——處理健康資料時這一點是紅線不是加分。

成本從哪裡來

一次 RAG 對話的 Token 組成



合計約 2,750 tokens ／次

召回段落佔 53%，是唯一會隨設定放大的部分；
System Prompt 每次呼叫都要重送一遍。

Top-K 調大，成本線性上升



Top-K 從 3 調到 10，單次 Token 幾乎翻倍；
調召回參數時，成本要跟準確度一起看。

★ System Prompt 每次呼叫都重送；召回段落是唯一會隨設定放大的部分。

實務策略：簡單的判定節點用小模型（快、便宜），文書生成才用大模型。

左・Token 組成

召回的知識庫段落通常是最大宗，而且它隨 Top-K 線性增加

System Prompt

每次呼叫都會重送。寫得越長，每一次對話都要付一次

右・Top-K 的成本

從 3 調到 10，Token 幾乎翻倍——這是調參數時要一起考慮的

實務策略

簡單的判定節點用小模型（快、便宜），文書生成才用大模型

圖：課程自製（Token 數為典型量級示意）　★ 原生圖形，可直接在 PowerPoint 編輯

Dify 介面的四個區域

編排 Orchestrate

- Chatbot：直接編輯 System Prompt 、開場白、建議問題、變數
- Workflow / Chatflow：拖拉節點、連線、設定每個節點的輸入輸出
- 右側可即時 Debug 測試，不必發佈就能試

知識庫 Knowledge

- 上傳文件、設定切塊方式、選擇嵌入模型
- ★ 召回測試：輸入問題就能看到會召回哪幾段、各自的分數
- 這是 W14 最重要的工具——不要跳過它直接改參數

工具 Tools 與設定 Settings

- 工具：內建工具庫、自訂 API 工具、 MCP 連接器
- 設定 → 模型供應商：填入你自己的 API 金鑰
- 設定 → 監控／ Tracing：看每次呼叫的思考步驟與 Token 消耗

匯入既有範例的方式：首頁 → 建立應用 → 匯入 DSL 檔案 → 選 .yml 。匯入後記得先在設定中填入金鑰，否則無法執行。

成本控制的五個手法

手法	做法	省多少
精簡 System Prompt	刪掉重複與冗詞；範例保留 2 個就好	每次呼叫都省
降低 Top-K	從 8 降到 5，若召回品質不變就採用	線性減少
模型混用	判定節點用小模型，生成才用大模型	判定節點可省 8–10 倍
限制對話歷史	只保留最近 3–5 輪，不要無限累積	長對話省很多
設最大迭代次數	Agent 設 5–10 次上限	避免失控的極端情況

> 課堂與專題的實務建議：先用免費額度把流程跑通，確認邏輯正確後再考慮效能與成本的優化。不要一開始就糾結成本。

金鑰管理與模型選擇

金鑰規範（評分項目，也是資安底線）

- 每位同學自行申請 API 金鑰，不共用、不寫進報告、不上傳公開儲存庫
- Dify 的金鑰存在「設定 → 模型供應商」，不會出現在匯出的 DSL 檔——所以匯出檔可安全繳交
- Google Gemini 、 Groq 有免費額度（ Groq 速度快、額度較寬）； OpenAI 需先儲值

模型混用策略

- 歷屆專題實例：危機等級判定切到 Groq + Llama 系列以解決 429/503 錯誤，文書生成才用大模型
- 低延遲在急救、危機處理場域比長文本能力更重要——這是設計決策，不是技術限制
- 教材 LTC3.0 資料夾內的三個 yml 就是同一應用的不同模型版本，可直接比較

【 Dify 範例】 Dify/LTC3.0 Accessment/ 台灣長照 3.0 Chatflow 的 Gemini 2.5 / Gemini Flash / Groq Llama 三個版本。

PART 04

System Prompt 四段式

Prompt Engineering

同一個模型，換一段 System Prompt，
可以從話多的陌生人變成守規矩的專業助理。

System Prompt 的四個組成

1 角色與身分

- 你是誰？服務對象是誰？
- 例：你是長照個管師的評估助理
- 明確的身分會收斂用語與專業度
- 同時暗示了哪些話題該拒絕
- 避免寫成「你是全能助手」
- 越具體，行為越可預測

2 任務與步驟

- 要做什麼？依什麼順序做？
- 階層式拆解： 1. 辨識 2. 檢索 3. 判級 4. 建議
- 明確編號能有效降低跳躍式推理
- 每一步寫清楚輸入與輸出
- ★ 抑制幻覺最有效的手法之一
- 歷屆專題講評反覆強調

3 限制與邊界

- 不可以做什麼？何時該拒絕？
- 只能依知識庫回答，找不到就說找不到
- 不得提供診斷、處方或劑量
- 必須標註「僅供參考」
- 高風險徵象的處理規則
- ★ 醫療場域這段最重要

第四個組成是「輸出格式」——見下一頁。四者缺一，行為就會開始漂移。

實例：同一個問題，兩種 Prompt

無約束對照組

Prompt	內容
無約束版	你是一個健康照護助理，請回答使用者的健康問題。
提問	我最近胸口悶，可能是什麼病?
回答	**可能是心絞痛、胃食道逆流或焦慮症。建議觀察是否伴隨… （自行推測疾病，未標示限制）**

四段式版

Prompt	內容
四段式版	角色＋任務步驟＋限制＋輸出格式（見教學 02 ）
提問	同左
回答	【建議儘速就醫】胸悶屬警訊徵象… 【衛教說明】… 【資料來源】… 以上僅供參考，請諮詢醫師

差在哪

左邊自行推測疾病（超出範圍）、沒有警訊標示、沒有來源、沒有免責

右邊做對了什麼

偵測到「胸悶」屬警訊 → 強制前置就醫建議；格式固定；標註限制

這不是模型變聰明了

同一個模型、同一個問題。差別只在 System Prompt

★ 課堂實作

教學 01 與教學 02 就是這組對照，請自己跑一次感受差異

【Dify 範例】 教學 01 - 衛教問答 Chatbot （無約束對照組） .yml 與 教學 02 - 衛教問答 Chatbot （四段式 System Prompt ） .yml

抑制幻覺的四個手法（一）：規範思考與示範輸出

① 階層式任務拆解

- 規範思考步驟：辨識症狀 → 檢索知識庫 → 判斷分級 → 生成建議
- 不給步驟， LLM 會直接跳到結論，中間的推理是事後補編的
- 歷屆化療副作用與 EMT 專題都驗證過此手法有效

② Few-shot 範例引導

- System Prompt 中放 2–3 個「輸入 vs 標準輸出」的完整範例
- 範例比規則有效：模型會模仿範例的結構、語氣與詳盡程度
- 醫療場域可用 SOAP 或 ISBAR 格式，輸出立刻結構化

這兩招處理的是「怎麼想」與「長什麼樣」。下一頁的兩招處理「不准說什麼」。

抑制幻覺的四個手法（二）：約束輸出與設下界線

③ 輸出格式約束

- 明確規定欄位：評估摘要 / 建議處置 / 即刻處置 Red Flags / 資料來源
- 強制「 Red Flags 」欄位，可確保高風險訊息不被埋在長段落中間
- 更保險的做法：格式交給「模板轉換」節點，不只靠 Prompt 要求

④ 找不到就說找不到

- 明確指示：知識庫中沒有相關內容時，必須回答「查無資料」，不得推論
- ★ 關鍵在給「替代動作」——只寫「不要編造」效力很弱
- 這一句能擋掉大部分知識庫外的幻覺，是 RAG 系統的必備條款

四招都做完，仍擋不住「數字算錯」。那必須靠程式碼節點，見後續實作章節。

Prompt 對什麼有約束力、對什麼沒有

要求類型	約束力	為什麼	保險做法
結構／格式	★ 強	模型擅長模仿範例的形狀	給 Few-shot 範例
語氣／角色	★ 強	身分設定會全面影響用詞	角色寫具體
拒絕範圍	中	多數時候有效，但會被巧妙提問繞過	加流程節點做前置判斷
數量（幾條、幾字）	弱	「不超過 15 字」經常被違反	用流程節點驗證後截斷
數值計算	★ 很弱	LLM 心算不可靠	★ 一律用工具查表或計算
事實正確性	弱	訓練資料決定， Prompt 改不了	接知識庫（ RAG ）＋標註來源

最後兩列是本模組最重要的原則：需要「保證」的東西不能只靠 Prompt ，必須用流程節點、工具或知識庫來實現。

四段式 System Prompt 骨架（醫療場域通用）

把這份骨架複製下來，逐段替換成你的專題內容

System Prompt 骨架

```
# 角色
你是〈服務對象〉的〈領域〉助理，服務對象為〈使用者身分〉。

# 步驟
1. 先閱讀【參考資料】中召回的段落。
2. 〈判斷／分類／比對〉。
3. 〈整理輸出〉。
4. 需要精確數值時，交由後續程式節點計算，不自行寫出數字。

# 限制
- 【參考資料】未提及的內容，一律回答「查無相關資料，建議〈替代管道〉」。
- 不得提供診斷、處方、藥物名稱或劑量建議。
- 不得依症狀推測特定疾病。
- 偵測到【警訊清單】時，於最前段標示【建議儘速就醫】。

# 格式
依序輸出：【評估摘要】【建議處置】【Red Flags】【資料來源】
結尾固定加上免責聲明。

# 範例（Few-shot）
放 2–3 組「輸入 → 標準輸出」的完整示範。

【參考資料】
{{#context#}}
```

PART 05

RAG 檢索增強生成

Retrieval-Augmented Generation

LLM 知道很多事，但它不知道你機構的規定、不知道 2025 年的新法規，
也不知道自己正在編造。 RAG 補的就是這三個缺口。

LLM 的三個知識缺口，RAG 各補一個

1 時效缺口

- 模型的知識停在訓練截止日
- 長照給付標準 2025 年才更新
- 縣市補助規定隨時在變
- 問它「現在的標準」會給舊答案

▶ RAG：把最新文件放進知識庫

- 更新文件即更新知識，不必重訓

2 專屬缺口

- 你機構的作業規範不在訓練資料裡
- 院內用藥準則、通報流程、表單格式
- 模型只能給「一般性」的答案
- 一般性答案在臨床上往往等於錯的

▶ RAG：把內部文件變成可檢索來源

- 答案就會依循你的規範而非通則

3 可追溯缺口

- 模型說得頭頭是道但你查不到出處
- 臨床人員無法核對，就無法採信
- 幻覺與正確答案長得一模一樣
- ★ 醫療場域最致命的問題

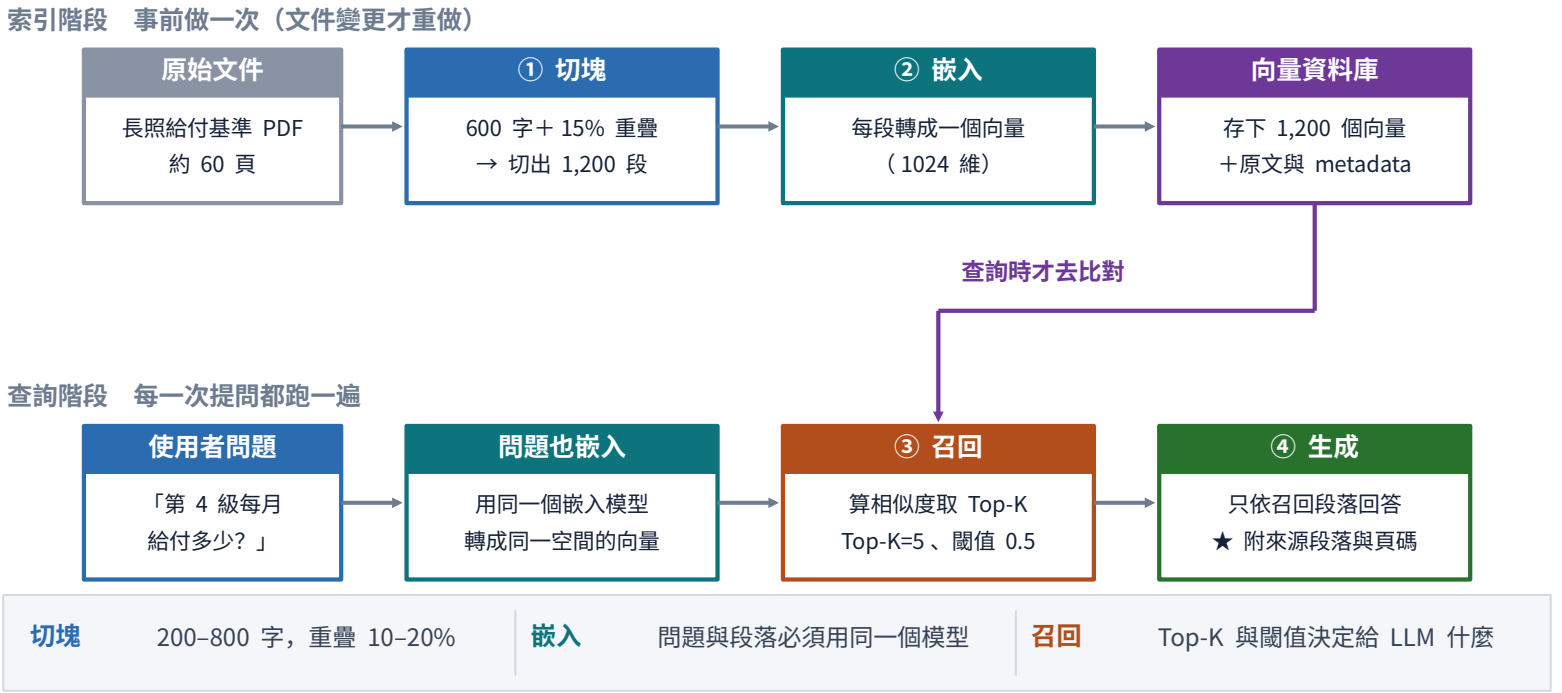
▶ RAG：答案可標註來源段落與頁碼

- 護理師能自己翻回原文確認

第三點最常被忽略，卻是醫療場域能否落地的分水嶺。歷屆化療副作用專題的講評第一條就是「強制標註知識庫原文段落或頁碼」。

RAG 的四個步驟

文件先切塊、嵌入、存進向量庫；提問時再比對取回最相近的幾段，交給 LLM 生成



★ 品質瓶頸幾乎都在第 ③ 步： LLM 只看得到「被召回的那幾段」。

召回錯了，生成再厲害也沒用——這也是為什麼一直改 Prompt 常常沒有效果。

除錯順序永遠是：先看召回了什麼、分數多少，再回頭調 Prompt 。

① 切塊 Chunking

把長文件切成小段落。切太大雜訊多、切太小斷語意

② 嵌入 Embedding

每段轉成向量存進向量資料庫。語意相近的段落，向量距離也相近

③ 召回 Retrieval

把問題也轉成向量，找出最相近的前 K 段

④ 生成 Generation

把召回的段落塞進 Prompt ，要求模型只依這些內容回答

圖：課程自製　★ 原生圖形，可直接在 PowerPoint 編輯

實例：切塊大小的取捨

下面三塊，是同一份《長照給付基準》用三種切法之後，知識庫裡實際存下的一段

太小　200 字
召回不完整

「CMS 第 4 級每月給付」

↑ 切點正好落在句子中間 ↑

「額度為 18,580 元，自負額…」

- 一條規定被切成兩半
- 問「第 4 級給付多少」只召回上半
- 下半段沒有「第 4 級」，比對不到

適中　600 字
★ 多數情況的預設

「CMS 第 4 級每月給付額度為 18,580 元，自負額依身分別分為一般戶 16%、中低收入戶 5%… 」

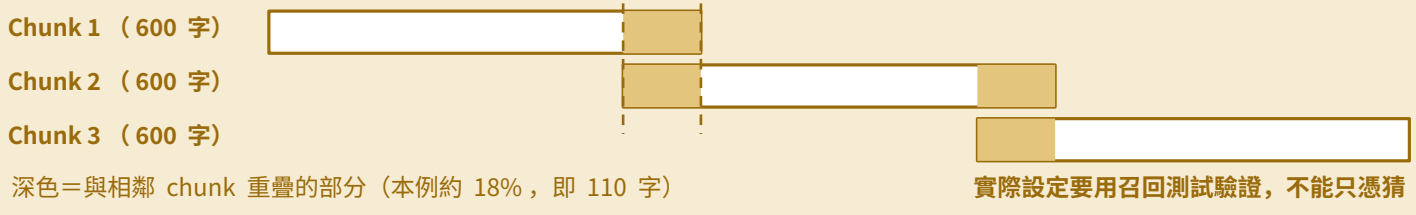
- 語意完整，一問就能整段召回
- 雜訊少、 Token 成本可控
- 條列式規定建議落在這一區

太大　1500 字
雜訊多、命中率降

「第三章　給付基準」（含全部 8 個等級、四類服務、各身分別自負額與申請流程…）

- 整章都在裡面，第 4 級只佔一小段
- Prompt 被無關內容塞滿，成本高
- 多段候選長得都很像，召回精準度降

重疊 Overlap：讓相鄰兩塊各留一段共同內容，切點兩側就不會有人「只拿到半句」



實際設定要用召回測試驗證，不能只憑猜

太小（200 字）
「CMS 第 4 級每月給付」與「額度 18,580 元」被切開 → 召回不完整

適中（600 字）
語意完整、雜訊少。多數情況的預設選擇

太大（1500 字）
整章都在裡面，雜訊多、 Token 成本高、召回精準度下降

重疊 Overlap
設 10–20% 可大幅降低「切在句子中間」的傷害

圖：課程自製　★ 原生圖形，可直接在 PowerPoint 編輯

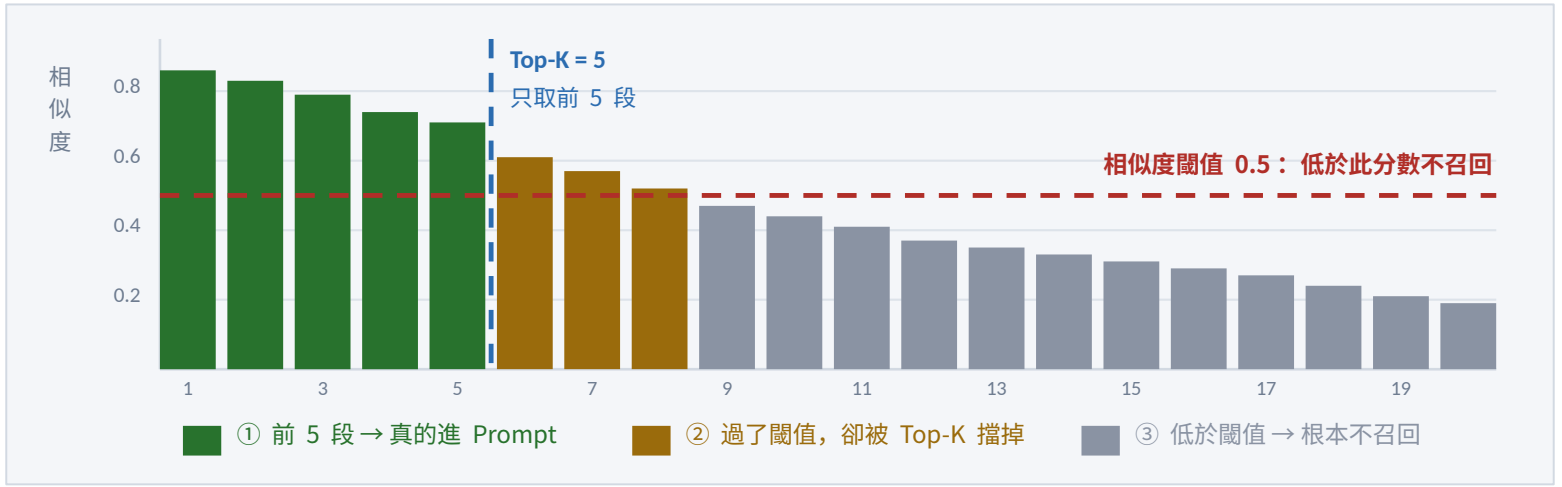
切塊策略對照

Chunk 大小	優點	風險	適合的文件
小（200–400 字）	召回精準； Prompt 雜訊少；成本低	容易切斷語意， 一條完整規定被拆成兩半	FAQ 、條列式規定、 術語表
中（500–800 字）	語意較完整；★ 多數情況的預設	偶爾仍會切斷長段落	一般政策文件、 衛教手冊
大（1000+ 字）	語意完整，跨段落關聯得以保留	雜訊多、 Token 成本高、 召回精準度下降	敘事性長文、 病例報告

長照給付基準這類條列式文件，建議用「小到中等 chunk ＋ 15% 重疊」。實際設定要用召回測試驗證，不能只憑猜。

召回設定： Top-K 與相似度閾值

20 個候選段落依相似度由高到低排序；兩個參數一起決定「哪幾段真的會進 Prompt」



Rerank 重排序

① 閾值放寬到 0.3
Top-K 開到 20

→

② Rerank 模型逐段重評
比向量相似度準

→

③ 取重排後的前 5 段
進 Prompt

★ Top-K 與閾值是「且」的關係：要進 Prompt，必須同時過閾值、又排進前 K。

怎麼調：拿 10 個真實問題做召回測試，看正確段落的分數落在哪一區，把閾值訂在它下面一點。

實務起點： Top-K 3–5 、閾值 0.5 。條列式規定可用 3；需要綜合判斷的用 5–8 。

檢索模式：法規條號、藥名這類專有名詞，用混合檢索（向量＋全文）比純向量可靠。

Top-K

召回幾段。太小可能漏掉正確答案，太大則塞入雜訊且成本上升

相似度閾值

低於此分數就不召回。防止「硬湊答案」的第一道防線

怎麼調

拿 10 個真實問題做召回測試，看正確段落的分數落在哪個區間

Rerank 重排序

先寬鬆召回 20 段，再用 Rerank 模型精選前 5 段——法規類文件效果特別明顯

圖：課程自製（分數為示意）　★ 原生圖形，可直接在 PowerPoint 編輯

三個一定要調的召回參數

Top-K （召回幾段）

- 太小：正確答案可能根本沒被召回，模型只能瞎猜或說查無資料
- 太大：塞進大量不相關內容，模型被雜訊帶偏， Token 成本同時暴增
- 實務起點： 3–5 。條列式規定可用 3 ，需要綜合判斷的用 5–8

相似度閾值（ Score Threshold ）

- 設太高會頻繁出現「查無資料」；設太低等於沒設
- 建議從 0.5 起調，用你自己的 10 個測試問題校準

檢索模式

- 向量檢索：懂語意，同義詞也找得到，但精確詞彙可能漏
- 全文檢索：精確比對關鍵字，法規條號、藥名這類專有名詞較可靠
- ★ 混合檢索 Hybrid： 兩者並用再合併排序——法規類知識庫建議用這個

知識庫的品質決定答案的品質

放什麼進去

- 法規、指引、院內規範、衛教手冊、標準作業流程——有權威來源的文件
- ★ 不要放：未經查證的網路文章、個人筆記、含個資的原始紀錄
- 文件本身要先整理：掃描檔需 OCR、表格要轉成文字或 CSV

怎麼組織

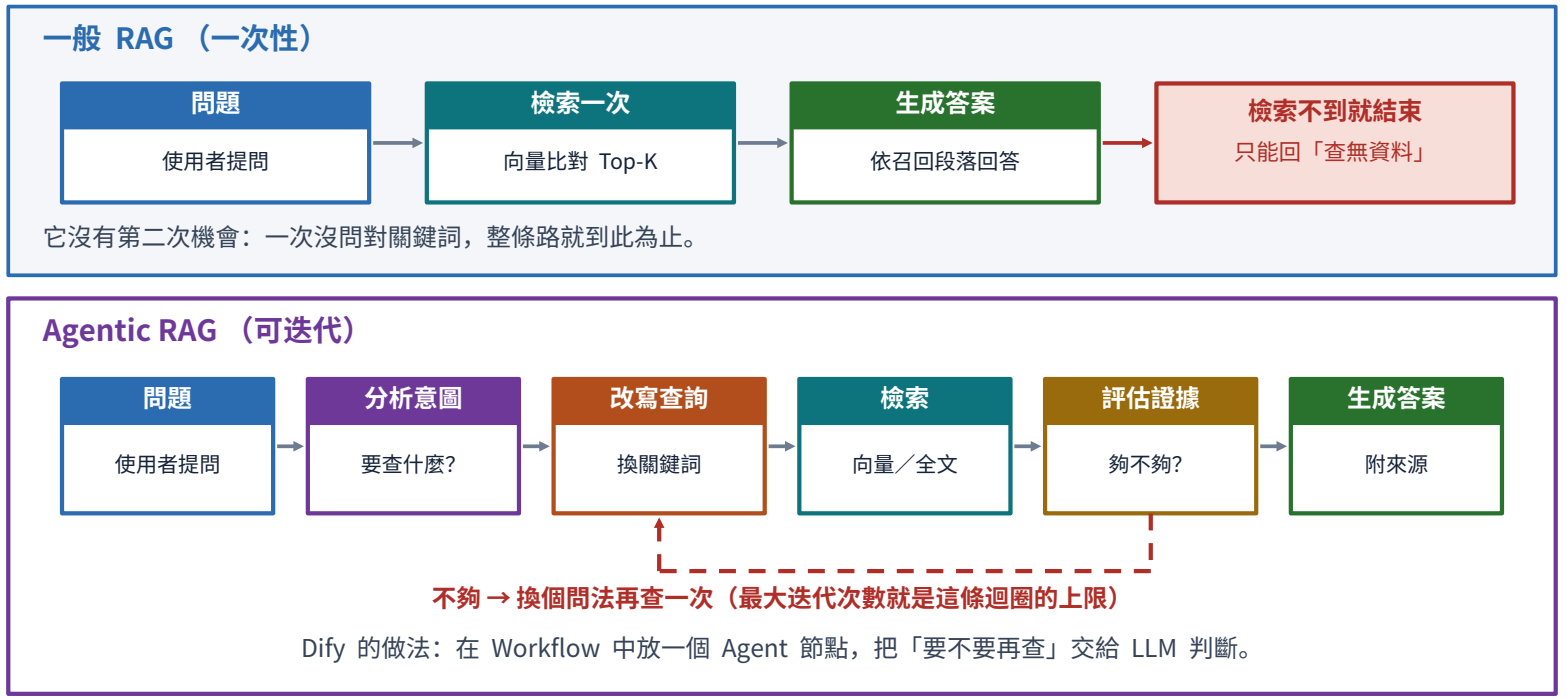
- 一個主題一個知識庫，不要把所有文件倒進同一個
- 用 metadata 標註縣市、年份、適用對象——之後可以做過濾檢索
- ★ 每份文件標註版本與生效日期，並要求輸出時一併顯示

怎麼維護

- 法規更新時要同步更新知識庫，並記錄更新日期
- 累積使用者問到但查不到的問題，作為補文件的清單
- 歷屆專題提醒： Firecrawl 等外部知識源可能因久未使用而停用，需定期檢查

2026 的新做法： Agentic RAG

差別只有一件事：檢索完之後，有沒有人回頭問一句「這些證據夠了嗎？」



★ 代價：迭代次數增加 → Token 成本與延遲都上升。務必設最大迭代次數。

一般 RAG 的成本可估算； Agentic RAG 不行——同一個問題，這次查 1 輪，下次可能查 4 輪。

值得的時機：問題本身就需要拆解，或單次檢索的命中率明顯不足時。

一般 RAG

檢索一次就生成。檢索不到就只能說查無資料

Agentic RAG

Agent 自己分析意圖、改寫查詢、評估證據夠不夠、不夠就換個問法重試

Dify 的做法

在 Workflow 中放 Agent 節點，讓 LLM 決定要不要再查一次

代價

迭代次數增加 → Token 成本與延遲都上升。要設最大迭代次數

圖：課程自製 ★原生圖形，可直接在 PowerPoint 編輯

RAG 的五種失效模式

五種失效在使用者眼裡都只是「答錯了」，但成因與處方完全不同

失效模式	看起來的症狀	Tracing 裡的證據	處方
召回不到	文件裡明明有，卻回答「查無資料」	知識檢索節點 召回 0 段	→ 降低 Score 閾值／加大 Top-K 檢查切塊是否把規定切散
召回錯的	答得很篤定，但引用的段落不相關	召回 5 段，分數都只有 0.3 左右	→ 加 Rerank ／提高閾值 清理知識庫裡的無關文件
召回到但沒用	正確段落有進 Prompt，答案仍然是編的	context 有內容，答案卻對不上	→ Prompt 強制「只依以下內容回答」；查無依據要說清楚
文件過時	正確地引用了錯誤的舊版規定	召回段落來自 2023 年版文件	→ 標註版本與生效日期並顯示 舊版文件下架或加註
跨文件矛盾	不同縣市／年份的規定同時出現	5 段來自 3 份不同文件	→ metadata 過濾：先依縣市／年份 把範圍縮小再檢索

★ 除錯的第一步永遠是打開 Tracing 看「召回了什麼」，而不是直接改 Prompt。

前兩種要調召回參數，第三種要調 Prompt，後兩種得治理知識庫本身（版本與 metadata）。

分不清是哪一種就亂調，常常是把原本會對的那一題也一起調壞。

召回不到 / 召回錯的

前者調閾值與 Top-K，後者加 Rerank 或清理知識庫

召回到了但沒用

正確段落有進 Prompt，答案仍是編的 → 強制「只依以下內容回答」

文件過時

正確地引用了錯誤的舊版規定 → 標註版本與生效日期並顯示

跨文件矛盾

不同縣市／年份同時被召回 → 用 metadata 過濾先縮小範圍

圖：課程自製　★ 原生圖形，可直接在 PowerPoint 編輯

隨堂練習：看 Tracing 證據，判斷是哪一種失效

1 知識檢索節點召回 0 段，但文件裡明明有，系統回答「查無資料」

召回不到

降低 Score 閾值／加大 Top-K；檢查切塊是否把規定切散

2 召回 5 段，分數都只有 0.3 左右；答得很篤定，但引用的段落不相關

召回錯的

加 Rerank／提高閾值；清理知識庫裡的無關文件

3 context 裡有正確段落，答案卻對不上

召回到但沒用

Prompt 強制「只依以下內容回答」；查無依據要說清楚

4 召回段落來自 2023 年版文件，正確地引用了舊版規定

文件過時

標註版本與生效日期並顯示；舊版文件下架或加註

5 5 段來自 3 份不同縣市的文件，規定互相矛盾

跨文件矛盾

用 metadata 過濾：先依縣市／年份縮小範圍再檢索

6 正確的給付段落排在第 9 名，Top-K = 5

召回不到

先看切塊（答案可能被切散），再考慮 Rerank 或調閾值

前兩種調召回參數，第三種調 Prompt，後兩種要治理知識庫本身。順序：先打開 Tracing 看「檢索節點給了什麼」，再看「LLM 拿到後做了什麼」（見第 56 張除錯決策樹）。

實作 1 ｜台灣長照 3.0 知識庫

【Dify 範例】　LTC3.0 Accessment

開啟檔案

```
範例 /Dify/M8. Dify 應用開發 /
└ LTC3.0 Accessment/
  ├ 台灣長照 3.0 Chatflow
  │    (Gemini 2.5 ChatFlow improved).yml
  ├ 台灣長照 3.0 Chatflow (Groq Llama).yml
  └ 台灣長照 3.0 Chatflow (Gemini Flash).yml

資料 /Dify/ 知識庫素材 /
  KB1_CMS_ 評估量表與判定標準 _2025 正式版 .txt
  KB3_ 長照給付額度與自負額標準 _2025 正式版 .txt
  LTC_Payment_Standard_2025_Full_AF.csv
```

預期結果

- 預設切塊常把給付級距表切散，需調整才能正確召回
- 正確段落應排在前 3 名；排在第 8 名代表設定要改
- 知識庫外的問題若仍給出具體數字，代表 Prompt 約束不足
- ▶ Groq 明顯較快但用詞較不精準； Gemini 較慢但答案較完整

操作步驟

1. 建立知識庫，上傳 KB1 與 KB3，先用預設切塊設定
2. 進入「文件」檢視實際切出來的段落——有沒有把給付級距表切散?
3. 用「召回測試」輸入：「CMS 第 4 級每月給付額度是多少? 」記錄正確段落排第幾
4. 調整 chunk 大小與重疊（試 400/15% 與 800/15%），重跑召回測試比較排名
5. 匯入 improved 版 Chatflow，掛上知識庫
6. 問 3 個知識庫內有答案的、 2 個知識庫外的問題，觀察是否誠實說查無資料
7. 換成 Groq Llama 版本問同樣問題，比較回答品質與回應速度
8. ★ 檢查給付金額是「召回來的」還是「模型生成的」

數字不生成：本模組最重要的一條規則

為什麼

- 給付金額、藥物劑量、風險分數這類具體數字，是幻覺傷害最大的地方
- 因為它看起來就像正確答案——沒有任何徵兆告訴你它是編的
- 使用者不會去查證每一個數字，尤其在急迫的臨床情境下

怎麼做

- 數字一律來自「檢索」或「工具計算」，不可由 LLM 生成
- ▶ Dify 做法：用知識庫召回原文、或用 Code 節點做計算、或串接查表 API
- 在 Prompt 中明確禁止：「不得自行計算或推估任何金額與劑量」

怎麼驗證

- 同一個問題問五次，看數字會不會變。會變的就是生成的
- 在輸出中強制標註來源段落，讓使用者能自己核對
- ★ 這是專題驗收的檢查項目之一

【Dify 範例】 CMS Workflow - case assessment-01.yml 是「結構化判定 + 知識庫檢索」的組合範例，可觀摩它如何把「判定」與「查表」分開處理。

隨堂檢核 1 ｜ Prompt 與 RAG

Q 1

你的 RAG 系統對知識庫外的問題也給出了具體數字，最可能是哪裡出問題?

Prompt 缺少「找不到就說找不到」的明確指示，模型退回用訓練資料回答。

Q 2

召回測試中，正確段落排在第 9 名。你會先調哪一個參數?

先看切塊——答案可能被切散了。切塊沒問題再考慮加 Rerank 或調閾值。

Q 3

為什麼法規類知識庫建議用混合檢索而不是純向量檢索?

條號、藥名這類精確詞彙，向量檢索可能因語意相近而找錯；全文檢索能精確比對。

Q 4

同一個問題問五次，給付金額有三種答案。這代表什麼?

數字是模型生成的，不是查表來的。必須改成用知識庫召回或工具計算。

PART 06

可操作實例：從零建一個 Chatflow

Hands-on: LTC Payment Estimator

以「長照給付試算」為例，逐節點示範每一個欄位怎麼填。
本節所有參數與程式碼都可直接複製使用。

七個步驟的全貌

長照給付試算 Chatflow： Step 1–5 課堂 40 分鐘可完成， Step 6–7 為課後練習

Step 1	**建立應用**	首頁 → 建立空白應用 → 選「 Chatflow 」→ 命名
Step 2	**拉入知識檢索節點**	點「＋」→ 知識檢索 → 選 KB1_CMS 評估標準／ KB3_ 給付額度 多路召回、 Top-K=5 、 Score 閾值 0.5 、開啟 Rerank
Step 3	**接上 LLM 節點**	模型選 gemini-2.5-flash ，貼入四段式 System Prompt ★ 上下文欄位綁「知識檢索 /result 」，不是把文字貼進 Prompt
Step 4	**加條件分支**	IF/ELSE → 條件： LLM 輸出含「警訊」→ 走高風險分支
Step 5	**加程式碼節點**	Python 依 CMS 等級查表算金額（不讓 LLM 算） 輸入變數綁 LLM 的等級輸出；函式 return 一個 dict
Step 6	**模板轉換**	固定四段輸出格式 ＋ 插入「資料來源」欄位（引用 code.amount ）
Step 7	**直接回覆 ＋ 測試**	右側 Debug 面板輸入測試個案，打開 Tracing 逐節點檢查

★ 每一步做完就在右側 Debug 跑一次——不要七步做完才測試，出錯時你會不知道是哪一步壞的。

先看全貌再動手

接下來每一步都會給你「填什麼、設多少」的具體參數。

★ 每步都要測

做完一步就在右側 Debug 面板跑一次。七步做完才測試，出錯時你不會知道是哪一步壞的。

預計時間

課堂 40 分鐘可完成 Step 1–5； Step 6–7 為課後練習。

★ 原生圖形，可直接在 PowerPoint 編輯

Step 2　知識檢索節點：實際要填什麼

設定欄位	本例填入的值	為什麼
知識庫	KB1_CMS 評估標準、 KB3_ 給付額度	兩份文件性質不同，各自建庫避免互相干擾
檢索模式	多路召回（ Multi-path Retrieval ）	同時查兩個庫，再統一排序
Top K	5	太小漏資訊；太大塞爆 context 又拉高成本
Score 閾值	0.5	低於此分數的段落不回傳，避免無關內容混入
Rerank 模型	開啟（ bge-reranker-v2-m3 ）	召回 20 段再精排取 5 段，準確度明顯優於直接取 5
查詢變數	{{#start.case_desc#}}	綁定開始節點的輸入變數

> 常見錯誤： Top K 設成 20 想「保險一點」，結果無關段落淹沒正確段落，答案反而變差。

Step 3　LLM 節點：完整可貼上的 System Prompt

四段式結構：角色 → 步驟 → 限制 → 格式

System Prompt （可直接複製）

```
# 角色
你是長照服務諮詢助理，服務對象為長照個案家屬與照顧服務員。

# 步驟
1. 先閱讀【參考資料】中召回的段落。
2. 判斷個案描述對應的 CMS 等級（第 2 級至第 8 級）。
3. 說明該等級可申請的服務類別。
4. 給付金額由後續程式節點計算，你「不要」自行寫出任何金額數字。

# 限制
- 【參考資料】沒有提到的內容，一律回答「查無相關資料，建議洽詢當地照管中心」。
- 不得提供醫療診斷、用藥建議。
- 不得自行推估任何金額、比例或天數。
- 個案描述不足以判定等級時，明確列出「還需要知道哪些資訊」，不要猜。

# 格式
依序輸出四段，每段加粗標題：
【判定等級】 【判定理由】 【可申請服務】 【資料來源】
結尾固定加上：本回覆僅供參考，實際給付以照管專員評估結果為準。

【參考資料】
{{#context#}}
```

Step 3 延伸　同一個 Prompt，三種寫法的實測差異

寫法	輸出樣態	問題
只寫「你是長照助理，請回答問題」	回答流暢但夾雜非長照 3.0 的舊制資訊，且會自己編金額	無約束 → 幻覺率最高（教學 01 對照組）
加上「# 限制：不要編造」	編造變少，但遇到知識庫沒有的內容仍會「合理推論」	否定式指令效力弱，要給明確替代動作
★ 四段式 + 明確替代動作	知識庫沒有的內容穩定回覆「查無相關資料」	格式一致、可稽核（教學 02 實驗組）

課堂實驗：用同一個問題「植物人狀態可以申請多少錢？」分別測三種寫法，比較五次回答的一致性。

Step 5　程式碼節點：金額不讓 LLM 算

輸入變數綁定 LLM 的等級輸出，輸出 amount 供下游使用

Python （ Dify 程式碼節點）

```
def main(cms_level: str) -> dict:
    # 照顧及專業服務給付額度（月，新臺幣元；以衛福部最新公告為準）
    TABLE = {
        "2": 10020, "3": 15460, "4": 18580, "5": 24100,
        "6": 28070, "7": 32090, "8": 36180,
    }

    # 只接受「第 4 級」「CMS 4」或單獨的「4」，避免抓到句中其他數字
    import re
    m = re.search(r"第\s*([2-8])\s*級|CMS[^0-9]{0,4}([2-8])|^\s*([2-8])\s*$", cms_level or "")
    if not m:
        return {"amount": 0, "level": "無法判定",
                "note": "個案資訊不足，無法判定等級，請洽照管專員"}

    lv = m.group(1) or m.group(2) or m.group(3)
    return {
        "amount": TABLE[lv],
        "level": f"第{lv}級",
        "note": "本金額為每月給付上限，實際核定以照管中心評估為準",
    }
```

為什麼這 20 行程式碼是本模組最重要的內容

LLM 算數學不可靠，且錯得很自然

- 問「第 4 級一個月多少」，LLM 可能答 18,580（對）也可能答 18,850（錯）
- 錯誤的數字看起來跟正確的一模一樣，使用者無從察覺
- 同一個問題問五次得到不同數字 —— 這就是「數字是生成的」的證據

查表是確定性的

- 同樣的等級永遠得到同樣的金額，可稽核、可測試、可單元測試
- 法規調整時只要改 TABLE 這 3 行，不用重寫 Prompt 也不用重測全流程

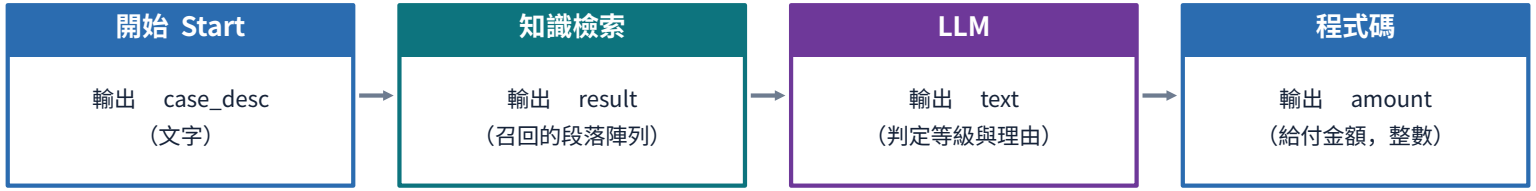
分工原則：LLM 做判斷，程式做計算

- LLM 擅長：從自然語言個案描述判斷「這像是第幾級」
- 程式擅長：查表、算術、格式驗證、邊界檢查
- ★ 把兩者混在一起，是初學者最常見也最危險的錯誤

推論延伸：任何「不能算錯」的數值（劑量、金額、天數、分數）都應該離開 LLM。

變數綁定：節點之間怎麼傳資料

① 連線：決定「誰先誰後」



連線畫好 ≠ 資料有傳過去 ↓ 每個下游節點還要自己把「輸入」綁到上游的哪個輸出

② 變數綁定：決定每個節點實際收到什麼

節點	要填的欄位	綁定來源（Dify 的引用語法）
知識檢索	查詢變數	開始／case_desc
LLM	上下文 Context	知識檢索／result ★綁欄位，不是把文字貼進 Prompt
LLM	Prompt 中引用	{{# 開始 .case_desc#}} —— 雙大括號＋井號才是引用
程式碼	輸入變數	LLM ／ text → 對應成 Python 參數名，函式 return 一個 dict
模板轉換	引用	{{# 程式碼 .amount#}} ★引用程式碼的輸出，不是 LLM 的 text

★ 最常見的失敗：變數沒綁 → 下游收到空值 → LLM 沒有資料可用 → 開始自由發揮。

看起來像幻覺，其實是接線問題。答案特別流暢、完全沒引用知識庫內容時，先去看綁定。

程式碼節點 return 的那個 dict，鍵名就是下游可以引用的變數名——名字打錯就等於沒接。

這是初學者卡最久的地方

節點連線畫好，不代表資料有傳過去。連線決定「順序」，
變數綁定才決定「內容」。

知識檢索的結果

要綁到 LLM 節點的「上下文」欄位，
不是把文字貼進 Prompt。

程式碼節點

在「輸入變數」區把上游輸出對應成 Python 參數名。
函式必須 return 一個 dict。

★ 最常見的失敗

變數沒綁 → 下游收到空值 → LLM 沒有資料可用 → 開始自由發揮。
看起來像幻覺，其實是接線問題。

★ 原生圖形，可直接在 PowerPoint 編輯

實作演練｜三個測試個案，驗證流程是否可靠

這三個個案分別測「正常」「輸入不足」「範圍外」三種情況

開啟檔案

測試個案 A （正常路徑）
78 歲女性，中風後右側偏癱，
無法自行如廁與沐浴，
家屬白天需上班。

測試個案 B （資訊不足）
老人家最近比較沒力氣。

測試個案 C （知識庫外）
請問我可以申請國民年金嗎?

預期結果

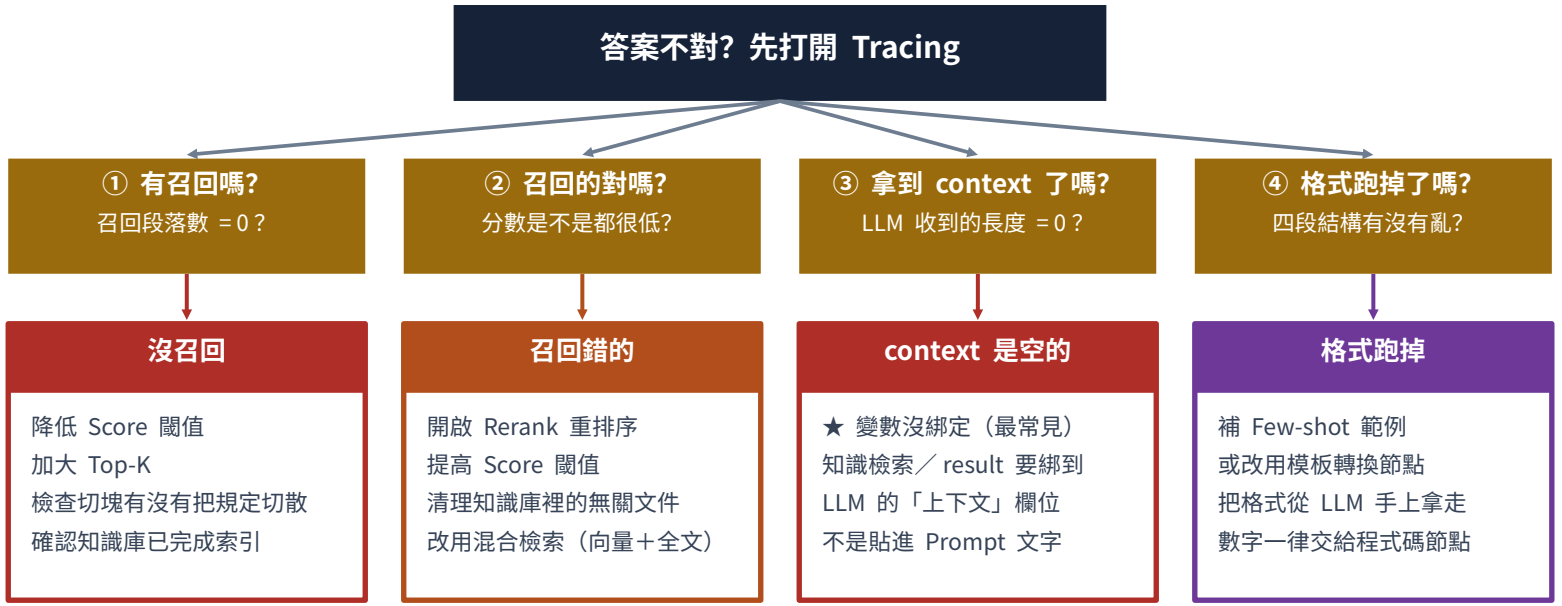
- A： 判定等級 ＋ 金額固定不變（來自查表）
- B： 明確列出缺少的評估項目，不給等級
- C： 查無相關資料 ＋ 建議洽詢管道
- 三個個案都帶「僅供參考」免責聲明

操作步驟

1. 在 Debug 面板依序輸入三個個案
2. 個案 A： 確認能判定等級、金額由程式碼節點產出
3. 打開 Tracing，逐節點檢查：知識檢索召回了幾段? 分數多少?
4. 個案 B： 確認回覆「還需要知道哪些資訊」而不是硬猜等級
5. 個案 C： 確認回覆「查無相關資料」而不是回答國民年金
6. 把個案 A 重複執行 5 次，比對金額是否每次相同
7. ★ 若 5 次金額不同 → 表示金額仍由 LLM 生成，程式碼節點沒接上

除錯決策樹：答案不對時，依序檢查什麼

由左而右，一格一格往下問——不要跳過前面直接去改 Prompt



★ 多數人一看到答案不對就改 Prompt—— 但如果 context 根本是空的，Prompt 改到天亮也沒用。

空 context 的辨識方法：答案完全沒引用知識庫內容、語氣特別流暢、沒有任何來源標註。

順序很重要

先確認「檢索有沒有給對東西」，再去調 Prompt。

多數人的錯誤反射

一看到答案不對就改 Prompt。但如果 context 根本是空的，
Prompt 改到天亮也沒用。

Tracing 要看的三個數字

召回段落數、各段相似度分數、LLM 實際收到的 context 長度。

空 context 的辨識方法

答案完全沒引用任何知識庫內容、語氣特別流暢、
沒有任何來源標註 → 幾乎可確定是變數沒綁。

★ 原生圖形，可直接在 PowerPoint 編輯

三個常見情境的完整診斷與處方

症狀	Tracing 看到什麼	處方
回答內容跟知識庫無關，但講得很順	知識檢索節點召回 0 段，或 LLM 的 context 欄位為空	① 檢查變數綁定 ② 降低 Score 閾值 ③ 確認知識庫已完成索引
每次問同一題，金額都不一樣	程式碼節點沒被執行，或輸出沒被下游引用	檢查模板轉換是否引用 code.amount ，而非 LLM 的 text
召回的段落把答案切成兩半	召回段落結尾正好斷在關鍵句中間	調整切塊：改用分隔符切塊、加大重疊字元數（ overlap ）
回答正確但格式每次都不同	LLM 節點輸出正常，但沒有模板轉換節點	加模板轉換節點固定格式，不要只靠 Prompt 要求格式
成本比預期高很多	Token 統計顯示 context 佔 80% 以上	降低 Top K 、開啟 Rerank 、縮短切塊長度

隨堂檢核｜模式選型與實作

Q 1

要做一個「把 200 份出院病摘轉成結構化表格」的工具，選哪種模式?

Workflow 。不需要對話、路徑固定、要能批次跑。用 Chatflow 反而多此一舉。

Q 2

Chatflow 裡，為什麼要用程式碼節點算給付金額，不直接讓 LLM 算?

LLM 產生數字是「生成」不是「計算」，同一問題多次執行可能給出不同數字，且錯誤數字看起來完全合理。查表是確定性的，可稽核、可測試。

Q 3

使用者問了知識庫裡完全沒有的問題，系統卻回答得頭頭是道。先檢查什麼?

先看 Tracing 的知識檢索節點：召回幾段、 context 是否為空。 context 空了 LLM 只能自由發揮。確認接線後，再回頭補 Prompt 的「查無資料」替代動作。

Q 4

什麼情況下 Agent 才真的比 Chatflow 好?

需求本身說不清楚要查什麼、需要組合多個外部來源、且允許路徑不可重現時。若你能畫得出流程圖，就該用 Chatflow 。

PART 07

Workflow 、 Chatflow 與 Agent

Choosing the Right Mode

專題一要求你論證「為何非用 Agent 不可」。
要論證得出來，得先知道另外兩個能做到什麼。

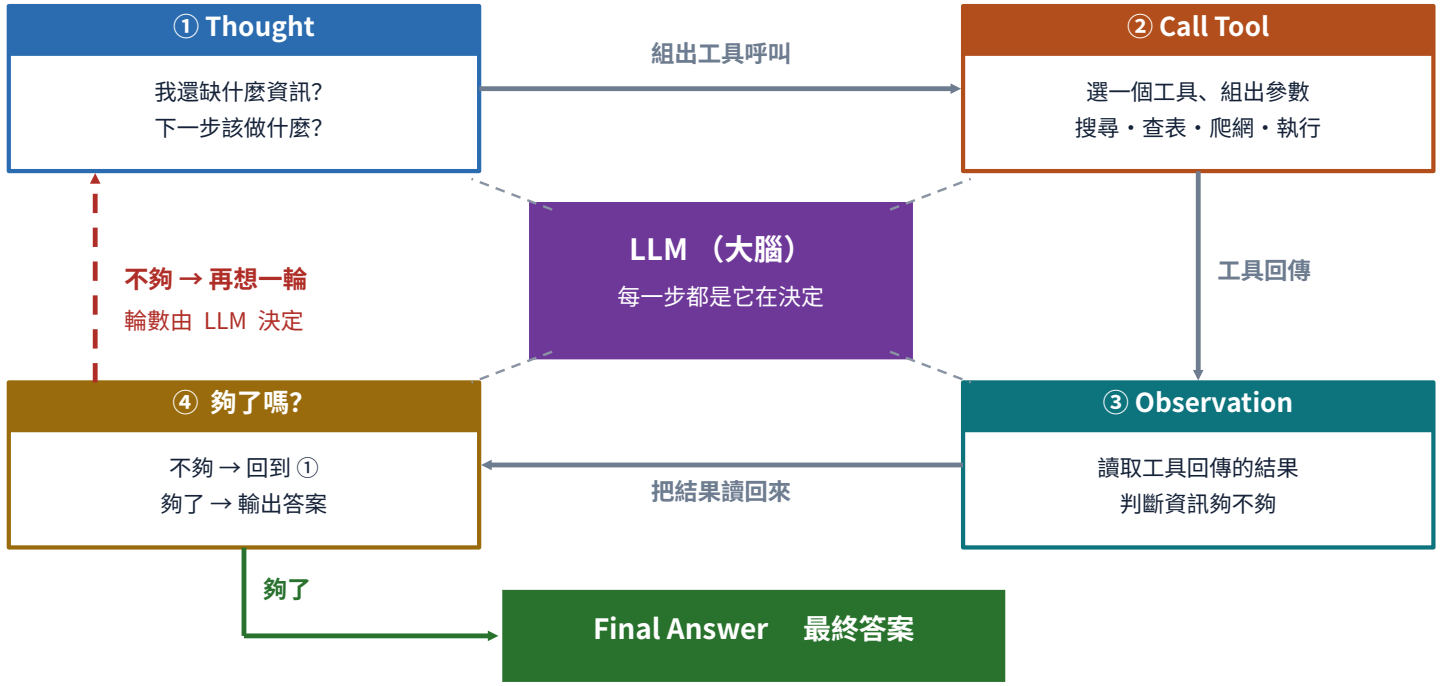
Workflow vs Agent

比較項目	Workflow （含 Chatflow ）	Agent
誰決定路徑	開發者事先決定，每次都一樣	LLM 每一步自己決定
同樣輸入跑兩次	Tracing 一模一樣	可能走不同路徑
除錯	出錯位置明確	需讀 Tracing 才知道它為何那樣做
適合的任務	步驟明確、可窮舉 （批次處理、報告生成）	任務模糊、需拆解 （多來源研究、複雜諮詢）
醫療場域的代價	低——行為可預測	高——需額外的約束與人為確認
Token 成本	可估算	不可估算（依迭代次數）

> 能用 Workflow 解決的就不要用 Agent。這句話會出現在專題評分裡——用 Agent 卻說不出理由的，「自主規劃能力」那 30% 會被扣。

Agent 的思考迴圈

每一輪都由 LLM 自己決定：還缺什麼、要不要再調一次工具、什麼時候可以收手



★ 務必設最大迭代次數：迴圈幾輪由 LLM 決定，沒有煞車就會一直燒 Token。

同樣的輸入，兩次執行可能迭代不同輪數——這就是 Agent 不可預測性的來源。

看 Tracing 時每一輪都要讀：它想了什麼、調了哪個工具、拿回什麼、為什麼決定再來一輪。

① Thought

LLM 判斷：現在缺什麼資訊？下一步該做什麼？

② Call Tool

選一個工具並組出參數（搜尋、查表、爬網頁、執行程式）

③ Observation

讀取工具回傳的結果，判斷資訊夠不夠

④ 迴圈或輸出

不夠就回到 ①，夠了才給 Final Answer。
★ 務必設最大迭代次數

圖：課程自製　★ 原生圖形，可直接在 PowerPoint 編輯

Workflow 常用節點

節點	做什麼	什麼時候用
開始 Start	定義輸入變數	所有流程的起點
LLM	呼叫模型生成內容	判斷、分類、摘要、生成
知識檢索 Knowledge Retrieval	從知識庫召回相關段落	RAG 的核心節點
條件分支 IF/ELSE	依變數值走不同路徑	★ 風險分流、資料完整性檢查
程式碼 Code	執行 Python ／ JavaScript	★ 數值計算、格式驗證、字串處理
Agent	讓 LLM 自主決定下一步	任務模糊、需組合多工具時
變數賦值 / 聚合	儲存中間結果、合併多分支	多輪對話的狀態管理
結束 End / 直接回覆	輸出最終結果	流程終點

設計原則：能用 Code 節點做的計算就不要交給 LLM 。 LLM 負責理解與生成， Code 負責精確運算——這個分工是可靠系統的基礎。

五種 Agent 類型（對應課程影片教材）

1 反射式 Simple Reflex

- 只看當下輸入，依規則直接反應
- 沒有記憶，不考慮歷史
- 例：關鍵字觸發固定回覆
- 最簡單，環境一變就失效
- 影片： 3 -Simple reflex agent.mp4
- 對應 Dify：純 Prompt 的 Chatbot

2 模型式／目標式

- 模型式：維護內部狀態，記得發生過什麼
- 目標式：有明確目標，會規劃達成路徑
- 開始有「為了達成 X 所以先做 Y」
- 多輪對話中收集資訊即屬此類
- 影片： 4 -Model-based / 5 -Goal-based
- 對應 Dify： Chatflow ＋ 變數記憶

3 效用式／學習式

- 效用式：多個可行方案時比較效用高低
- 學習式：從回饋中調整自己的判斷
- 例：在多個工具間權衡成本與準確度
- Dify 的 Agent 節點接近 目標＋效用
- 影片： 6 -Utility-based / 7 -Learning
- 學習式需外部回饋機制， Dify 需自建

教材位置： Generative AI/Video Clips/ 共 7 支影片與配套 pptx 。建議課前看完，課堂只討論它們與 Dify 實作的對應關係。

Agent 可用的工具類型

類型	例子	用途	注意
搜尋類	Google Search 、 Tavily 、 DuckDuckGo	取得即時網路資訊	結果品質不穩；需要驗證
知識類	Wikipedia 、 Arxiv	結構化的背景知識	不含最新或在地資訊
處理類	WebScraper 、 Code Interpreter	抓網頁、執行 Python 計算	★ 涉及數值計算一律用它
外部 API	以 OpenAPI 規格接任何 REST API	查藥物交互作用、給付碼對照	需處理認證與速率限制
MCP 連接器	檔案系統、資料庫、瀏覽器	標準化的外部連接	Dify v1.6 （2025.7）起支援雙向 MCP

> 工具描述要寫清楚—— LLM 是靠「描述」決定何時調用它。描述模糊就會在錯的時機亂調用，這是 Agent 最常見的失敗原因。

MCP： 把外部工具接給 AI 用

MCP 是什麼

- Model Context Protocol， Anthropic 在 2024 年提出的開放標準
- 讓 AI 助理以統一方式連接外部資料源與工具——資料庫、檔案系統、 API 、瀏覽器
- 類比： USB 之於周邊裝置。在此之前每個工具都要各自寫接法

2025 年起的進展

- Dify v1.6 （2025 年 7 月）起支援雙向 MCP： 既可當 client 連接外部 MCP 伺服器，也可把自己的應用開放為 MCP 服務
- 生態擴張很快，但成熟度不一——導入前先確認該連接器的維護狀態

為什麼你已經懂它了

- 這跟本模組的「工具調用」是同一個概念，只是換了標準化的接法
- M10 Vibe Coding 會再遇到 MCP （讓編碼助理連上檔案系統與資料庫）
- 共同原則：工具描述要寫清楚，否則 AI 會在錯的時機調用它

MCP 生態變化快，實際可用的連接器請以官方文件為準。本課程只要求理解概念，不要求實作 MCP 伺服器。

實作 2 ｜ Agent 與工具調用

【Dify 範例】 Component-Agent / Movie Researcher / 教學 03

開啟檔案

```
範例 /Dify/M8. Dify 應用開發 /
├ Marketing Plan Generator.yml
│    ← Workflow 對照組（路徑固定）
├ Component-Agent-01.yml  基礎 Agent
├ Component-Agent-02.yml  多元件組合
├ 教學 03 - 健康主題查證 Agent.yml
│  ← 含工具失敗處理與來源標註
├ Movie Researcher (Wiki).yml  單工具
└ Movie Researcher (Tavily).yml  網路搜尋

參考: Generative AI/Video Clips/
  7 支 Agent 類型教學影片
```

預期結果

- Workflow 每次 Tracing 相同； Agent 每次不同
- 工具描述被刪除後， Agent 可能完全不調用工具而直接用內建知識回答
- 迭代輪數越多、 Token 成本越高——這是 Agent 的隱藏成本
- ★ 第 8 步的截圖是期末專題的必要材料，請及早蒐集

操作步驟

1. 先跑 Marketing Plan Generator （Workflow），跑兩次比較 Tracing——應一模一樣
2. 匯入 Component-Agent-01 （Agent），跑兩次比較 Tracing—— 路徑很可能不同
3. 匯入教學 03，問一個 Wikipedia 上有的主題，數它調用幾次工具、迭代幾輪
4. 問一個查不到的冷門主題，觀察它是誠實回報還是開始編造
5. ★ 實驗：刪掉 Prompt 中「不得使用你自己的記憶補充」這條，重問並比較
6. 在 Agent 節點設定中把「最大迭代次數」改成 2，觀察任務未完成時的行為
7. 比較 Movie Researcher 的 Wiki 版與 Tavily 版對「上個月新片」的處理差異
8. 把你的專題場景套進教學 03 架構，跑複雜情境並截圖 Tracing

讀懂 Tracing：五個要看的欄位

欄位	看什麼	異常的徵兆
節點執行順序	流程實際走了哪條路	走了你沒預期的分支 → 條件設錯
各節點輸入／輸出	變數有沒有正確傳遞	下游節點收到空值 → 變數綁定錯誤
召回結果	召回了哪幾段、分數多少	正確段落沒被召回 → 調切塊或閾值
Agent 的思考過程	Thought → Call Tool → Observation	沒有調用工具 → 工具描述太模糊
Token 消耗與耗時	哪個節點最貴、最慢	單一節點暴增 → 檢查 Top-K 或 Prompt 長度

★ 期末專題要求附上 Tracing 截圖，具體展示 Agent 在某次測試中的完整軌跡（Thought → Call Tool → Output → Final Answer）。請及早熟悉這個介面。

PART 08

異常處理與人在迴路

Failure Handling & HITL

流程不會永遠順利。

專題評分的「邏輯穩定性」 20% 評的就是這一節。

會出什麼錯，該怎麼接

實際會遇到的失敗

- 工具調用失敗： API 額度用盡（429）、服務不可用（503）、逾時
- 回傳格式改變：外部 API 改版，解析失敗
- 模型拒絕回覆：內容被安全機制攔截
- 知識庫回傳空結果：查無相關段落
- 使用者輸入不足：資訊不夠做出判斷
- 迭代次數用盡： Agent 繞不出來

該有的處理

- 每個工具節點配一條失敗分支，不要讓流程靜默中斷
- 備援模型：主模型失敗時自動切換供應商（歷屆專題用 Groq 解 429/503）
- 備援資料源：網路搜尋失敗退回內建知識庫，★ 並明確告知使用者
- 資訊不足時反向提問，附上具引導性的問題清單
- 設定最大迭代次數，避免無限循環燒 Token
- 加入異常回報節點，引導使用者聯繫真人

歷屆聽損輔具專題的講評重點：當 Search Tool 失效改用內建知識庫時，前端要明確標示「目前使用內建法規庫，非即時資訊」——使用者的決策必須建立在知道限制的前提上。

隨堂練習：這個失敗該怎麼接?

1 主模型回傳 429 （API 額度用盡）

失敗分支＋備援

自動切換供應商——歷屆專題用 Groq 解 429/503

2 網路搜尋工具失效

退回內建庫並告知

前端明確標示「目前使用內建法規庫，非即時資訊」

3 使用者只輸入「老人家最近比較沒力氣」

反向提問

附上具引導性的問題清單，不要硬猜

4 Agent 一直換關鍵詞重查，繞不出來

最大迭代次數

設上限（5–10 次），避免無限循環燒 Token

5 模型拒絕回覆（內容被安全機制攔截）

轉真人

加入異常回報節點，引導使用者聯繫真人

6 知識庫回傳空結果，問題又涉及個案權益

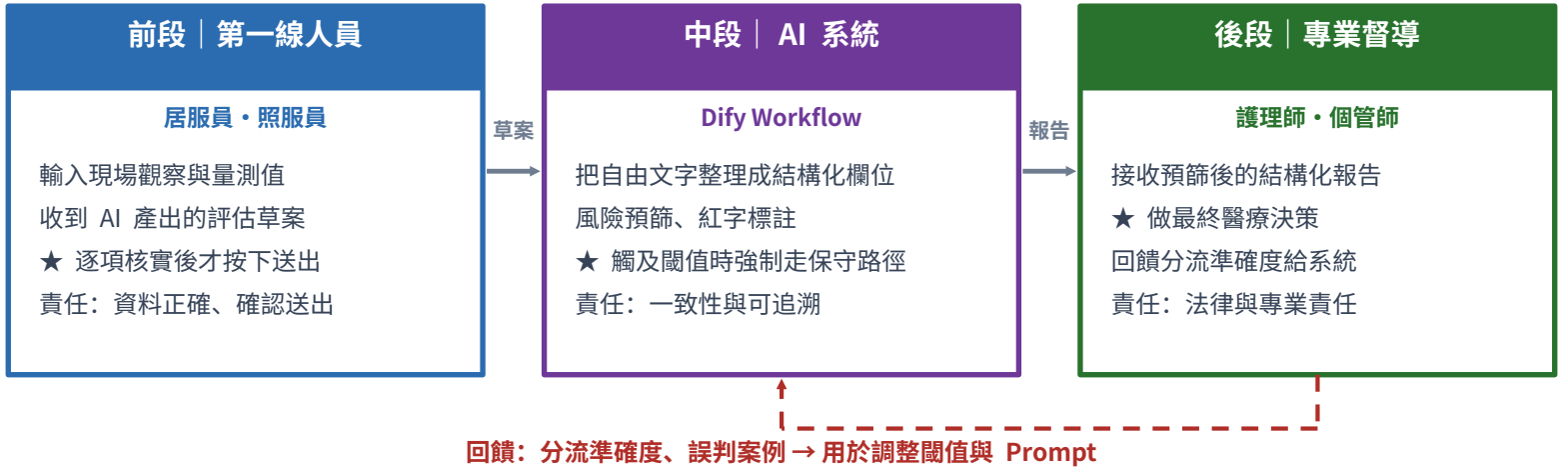
轉真人

誠實回「查無相關資料」，並提供照管中心等聯絡管道

原則：每個會失敗的節點都要有一條失敗分支，不讓流程靜默中斷；外部 API 改版、格式解析失敗也一樣走失敗分支。專題評分會看你有沒有設計這些。

Human-in-the-Loop： 三段式權責分工

AI 站在中間，但送出與拍板都在人手上——這條分界線決定了系統能不能落地



★ 三個一定要有人的地方

送出前	高風險時	最終決策
第一線人員逐項核實草案	強制紅字並通報督導	必須是有執照的專業人員

★ 為什麼一定要有人在迴圈裡： Agent 的自主性代表不可預測，而醫療決策的錯誤成本無法承受。

法律與專業責任無法轉嫁給系統——最終決策者必須是有執照的專業人員。

系統的定位是「把人的注意力放到對的地方」，不是「代替人做決定」。

前段・第一線人員

輸入現場觀察，並對 AI 產出的草案執行「人為核實」後才送出

中段・ AI 系統

結構化、風險預篩、紅字標註；觸及閾值時強制走最保守的建議路徑

後段・專業督導

接收預篩後的結構化報告，做最終醫療決策，並回饋分流準確度

為什麼必要

Agent 的自主性代表不可預測，而法律與專業責任無法轉嫁給系統

圖：課程自製　★原生圖形，可直接在 PowerPoint 編輯

實例：居服員 ISBAR 通報——一則現場觀察走完三段

前段・第一線人員　居服員・照服員	中段・AI 系統　Dify Workflow	後段・專業督導　護理師・個管師
一般觀察 「王奶奶今天午餐吃了半碗，下午在客廳看電視，精神還可以，血壓 128/82。」 送出前逐項核實 AI 草案： ☑ S ☑ A ☑ R	**S 現況：** 精神可，午餐進食約一半 **A 評估：** 生命徵象在常規範圍；進食量略少 **R 建議：** 持續觀察進食量，下次訪視追蹤 僅供參考，最終應由專業人員判斷。	收到預篩後的結構化報告 **決策：列入例行追蹤** 回饋分流準確度 → 用於調整閾值與 Prompt
含高風險徵象 「王奶奶下午從床邊跌倒，叫她時反應比平常慢，右手腕腫，血壓 168/96。」 送出前逐項核實 AI 草案： ☑ S ☑ A ☑ R	**【即刻處置 Red Flags】跌倒＋意識改變 → 強制保守路徑，立即通報督導** **S 現況：** 下午自床邊跌倒，反應較平常遲緩，右手腕腫脹 **A 評估：** 血壓 168/96 偏高；疑似頭部或腕部受傷 **R 建議：** 請護理師立即評估；必要時送醫 僅供參考，最終應由專業人員判斷。	🔴 **高風險通報已置頂** 收到預篩後的結構化報告 **決策：立即到宅評估／安排送醫** 回饋分流準確度 → 用於調整閾值與 Prompt

三個一定要有人的地方：送出前（第一線人員逐項核實）、高風險時（強制紅字並通報督導）、最終決策（必須是有執照的專業人員）。法律與專業責任無法轉嫁給系統。（虛構個案）

落實 HITL 的四個具體做法

做法	怎麼實作	效果
確認節點	高風險輸出必須經使用者點擊確認才進入下一步	建立法律與專業上的「人為確認」機制
強制紅旗欄位	輸出格式中固定包含「即刻處置 Red Flags」並置於最前段	高風險訊息不會被埋在長段落中間
安全降級	偵測到風險指標時，強制走最保守的建議路徑	例：偵測跌倒風險 → 運動處方自動降為坐姿
固定免責標註	所有醫療類輸出結尾標註「僅供參考，最終應由專業人員判斷」	明確界定系統的定位與責任邊界

> 這四項在期末專題若涉及醫療場域，是驗收的檢查項目。第 16 週的 M9 倫理模組會再深入。

PART 09

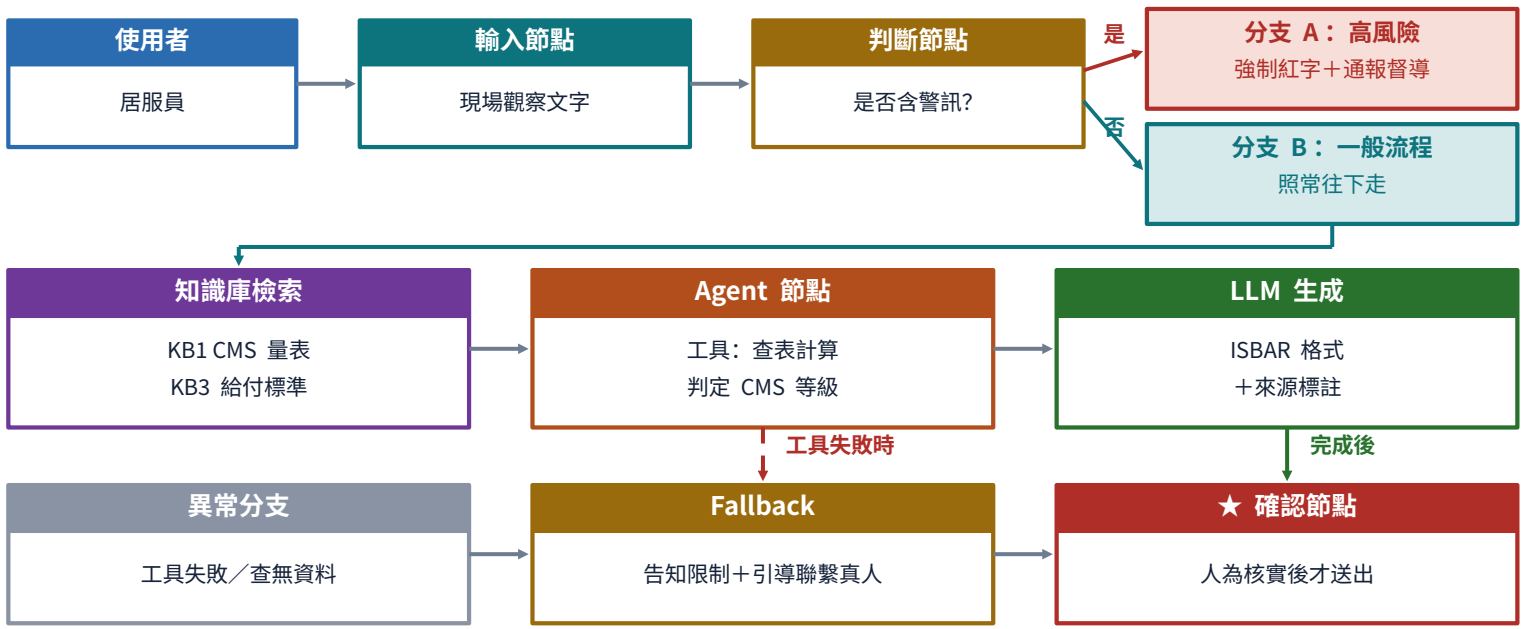
期末專題

Final Project: Dify Agent

構想書 W15 繳交（10%）｜實作 Demo W16 發表（20%）

合計佔學期成績 30%

專題架構藍圖範例



★ 這張圖就是專題構想書要交的「系統架構藍圖」。

節點、工具、知識庫、分支、 Fallback 、人為確認點——六樣都要標出來，缺一項就會被要求補件。

每個會失敗的節點（工具調用、知識檢索、外部 API ）都要有一條失敗分支，不能假設它一定成功。

最後一個節點永遠是「人為確認」：系統可以擬稿，但送出的責任在人。

上排

使用者輸入 → 判斷節點 → 依風險分支。這是流程的主幹

中排

知識庫檢索 → Agent 節點（工具調用）→ LLM 生成結構化輸出

下排

異常分支 → Fallback → ★ 人為確認節點

這就是構想書要交的圖

節點、工具、知識庫、分支、 Fallback 、確認點都要標出來

圖：課程自製　★原生圖形，可直接在 PowerPoint 編輯

第一階段：構想書（W15 繳交，10%）

項目	要求
篇幅	2–3 頁 PDF
1 問題定義與目標對象	Agent 要解決什麼實際問題？主要服務對象是誰？ 設定一個「複雜任務」情境（不只是回答定義）
2 為何選擇 Agent 模式	★ 論述為何此任務需要「自主行動與工具調用」， 而不是單純 Chatbot 或路徑固定的 Workflow
3 系統架構藍圖	底層 LLM 選擇與理由；工具清單與用途； System Prompt 草稿；流程圖（比照上一頁）
4 風險與限制預想	可能的失敗模式、 Fallback 設計、 若涉及醫療場域須說明 HITL 設計

第二階段：實作與 Demo （ W16 發表， 20% ）

項目	要求	評分佔比
可測試的 App URL	透過 Dify 發佈為獨立網頁，繳交公開連結	—
DSL 匯出檔	含完整 Agent 設定、 System Prompt 、流程佈局。 ★ 不含金鑰，可安全繳交	—
自主規劃能力	至少一個複雜情境測試案例，證明 Agent 能「任務拆解」	30%
工具調用準確性	有效調用外部工具取得即時資訊，而非僅依賴模型內建知識	30%
邏輯穩定性	流程順暢；★ 有處理工具失效的備案與提示	20%
解說完整度	Tracing 截圖（ Thought → Call Tool → Output ）； Prompt 工程說明；實作反思	20%

報告必備段落：提示詞工程說明（設了哪些限制以避免幻覺）、工具整合說明（串接了什麼、遇到什麼報錯）、實作反思（成果與構想的落差、你怎麼調整）。

選題建議：歷屆做過什麼

臨床照護類

- 化療副作用照護決策支援—— RAG 來源引用、 CTCAE 分級、多重症狀平行處理
- EMT 救護處置助理——屬地法規強制約束、線上醫療指導授權、 ePCR 草案生成
- 智慧復健姿勢回饋—— Patient-aware 判斷、 VAS 疼痛量表閾值、安全中止流程

長照與社區類

- 高齡運動處方生成（ ICOPE + FITT ）——風險自動降級、緊急停止指標前置
- 居服員 ISBAR 通報——三位一體權責分工、人為核實、督導回饋回流
- 聽損輔具補助導航——縣市政策動態權重、 Fallback 透明標示、一鍵產出申請清單

教育與公衛類

- 校園心理危機守門員——保護因子權重、對家長談話劇本、客觀事實與專業評估分離
- 共同特徵：都是第一線人員每天遇到、目前靠經驗與紙本處理的問題
- ★ 你的貫穿主線題目應該在第 1 週就選定，這裡只是把它做成 Agent

專題自我檢核（發表前逐項確認）

面向	檢核項目
自主規劃 30%	☐ 有一個「複雜情境」測試案例　☐ Agent 能拆解模糊指令 ☐ Tracing 顯示多輪 Thought → Call Tool
工具調用 30%	☐ 實際調用外部工具取得即時資訊　☐ 工具描述寫得具體 ☐ 數字來自查表或工具計算，非 LLM 生成
邏輯穩定 20%	☐ 每個工具節點都有失敗分支　☐ 有 Fallback 且告知使用者 ☐ 設了最大迭代次數　☐ 資訊不足時會反向提問
解說完整 20%	☐ Tracing 截圖完整　☐ 說明 Prompt 中設了哪些限制 ☐ 說明遇到什麼報錯與如何解決　☐ 有實作反思
醫療場域必要項	☐ 有免責標註　☐ 有來源標註　☐ 有人為確認節點 ☐ 高風險徵象有強制前置提示

最後一列若涉及醫療或教育場域，視為必要而非加分； M7 的四條紅線不符即扣分。第 16 週的 M9 會以此為基礎討論責任邊界。

Dify 應用開發的八個常見錯誤

#	錯誤	後果
1	System Prompt 只寫一句「你是助理」	行為漂移、超出範圍、格式不一致
2	知識庫直接用預設切塊就上線	表格被切散，關鍵數字召回不到
3	沒做召回測試就開始調 Prompt	調錯地方——問題在檢索不在生成
4	讓 LLM 計算金額或劑量	數字幻覺，且看不出來
5	工具節點沒有失敗分支	API 掛掉時靜默給出錯誤答案
6	Agent 沒設最大迭代次數	極端情況下無限循環燒 Token
7	備援啟動時沒告知使用者	使用者以為是即時資訊，實際是舊的內建知識
8	醫療輸出沒有免責與來源標註	無法核對、無法究責，臨床不會採用

隨堂檢核 2 ｜ Agent 與專題

Q 1

你的 Agent 每次都走同樣的路徑，這樣算 Agent 嗎?

路徑固定代表任務其實是可窮舉的——那用 Workflow 更好。這會影響「自主規劃能力」那 30%。

Q 2

Agent 完全不調用工具，直接用內建知識回答。最可能的原因?

工具描述寫得太模糊，或 System Prompt 沒有明確要求「必須查證」。

Q 3

工具回傳 429 錯誤，你的流程會怎麼樣?

沒有失敗分支的話會靜默中斷或給出錯誤答案。這是「邏輯穩定性」 20% 的檢查點。

Q 4

個管師問「這個給付金額哪來的」，你的系統答得出來嗎?

答不出來就是沒做來源標註。醫療場域這是落地的分水嶺，不是加分項。

本模組 Dify 範例總覽

週次	範例檔	用途
W13	教學 01 （無約束）／教學 02 （四段式）衛教問答 .yml	★ 對照組：同一問題、兩種 Prompt
W13	AI Chatbot (healthcare).yml	基礎 Chatbot ＋ 知識庫
W13	Slogan Generator-1 / -2.yml	變數、輸入欄位與輸出約束的對照
W13	Company Research.yml	結構化輸出
W14	LTC3.0 Accessment： 三個 Chatflow yml	★ 同一應用的三種模型版本，可直接比較
W14	CMS Workflow - case assessment-01.yml	結構化判定 ＋ 知識庫檢索的組合
W15	Marketing Plan Generator.yml	Workflow 對照組（路徑固定）
W15	Component-Agent-01 / -02.yml	Agent 基礎元件與多元件組合
W15	教學 03 - 健康主題查證 Agent.yml	★ 含工具失敗處理與來源標註
W15	Movie Researcher (Wiki) / (Tavily).yml	單工具 vs 多工具對照

全部位於「範例 /Dify/M8. Dify 應用開發 / 」。教學 01–03 為本課程新製作，其餘為原課程教材。

中英術語對照｜ M8

（1/4）

中文	English	一句話說明
大型語言模型	Large Language Model, LLM	以大規模文本預訓練的語言模型
提示詞	Prompt	給模型的指令與上下文
系統提示詞	System Prompt	設定角色、任務、限制與輸出格式的底層指令
提示詞工程	Prompt Engineering	以指令設計引導模型行為的技術
少樣本提示	Few-shot Prompting	在提示中給 2–3 個範例引導輸出格式
零樣本提示	Zero-shot Prompting	不給範例，直接下指令
思維鏈	Chain of Thought, CoT	要求模型逐步推理再給結論
幻覺	Hallucination	模型產生看似合理但不正確的內容
Token	Token	模型處理的最小文字單位；中文約 0.6–1.5 個 token ／字，依模型而異

中英術語對照｜ M8

（2 / 4）

中文	English	一句話說明
上下文長度	Context Length / Window	單次呼叫可處理的 Token 上限
溫度	Temperature	控制輸出隨機性；低則穩定、高則多樣
檢索增強生成	Retrieval-Augmented Generation, RAG	先檢索文件再生成，可標註來源
切塊	Chunking	把長文件切成可檢索的小段落
重疊	Overlap	相鄰切塊間重複的比例，避免切斷語意
嵌入	Embedding	把文字轉成向量表示
向量資料庫	Vector Database	儲存並快速檢索向量的資料庫
召回	Retrieval	依相似度取出最相關的段落
Top-K	Top-K	召回段落的數量上限

中英術語對照｜ M8

（3 / 4）

中文	English	一句話說明
相似度閾值	Score Threshold	低於此分數不予召回
重排序	Rerank	對召回結果再做一次精細排序
混合檢索	Hybrid Search	向量檢索與全文檢索並用
代理式檢索	Agentic RAG	由 Agent 自行判斷是否改寫查詢並重試
代理	Agent	能自主規劃並調用工具完成任務的系統
工具調用	Tool Calling / Function Calling	模型依需求呼叫外部功能
思考迴圈	Thought–Action–Observation Loop	Agent 反覆思考、行動、觀察的循環
最大迭代次數	Max Iterations	限制 Agent 迴圈次數，避免無限循環
模型情境協定	Model Context Protocol, MCP	連接 AI 與外部工具／資料源的開放標準

中英術語對照｜ M8

（4 / 4）

中文	English	一句話說明
工作流	Workflow	路徑固定的自動化流程
對話流	Chatflow	含對話與條件分支的流程
領域特定語言檔	DSL (Domain-Specific Language) 匯出檔	Dify 應用的可攜設定檔（.yml）
追蹤日誌	Tracing	記錄每一步思考、調用與輸出的日誌
備援	Fallback	主要路徑失效時的替代方案
人在迴路	Human-in-the-Loop, HITL	關鍵決策點保留人為確認與最終判斷
安全降級	Safe Degradation	偵測到風險時強制走最保守的建議路徑
來源標註	Source Citation	在輸出中標明引用的原文段落，供人核對

PART 10

不只有 Dify：工具全景與選型

The Agent Tooling Landscape

Dify 只是這個生態的一塊。 n8n 、 Flowise 、 Coze 各自解決不同的問題。
選錯工具，事倍功半；選對了，一個下午就能做完。

先釐清一個常見誤解

「n8n 和 Dify 是競爭對手」——不完全對

- 兩者出身不同：n8n 是「工作流自動化平台」，AI 是後來加上的能力
- Dify 是「LLM 應用開發平台」，從第一天就是為了 AI 而生
- ★ 它們解決的是不同的問題，實務上經常「一起用」而不是「二選一」

判斷的關鍵問題：AI 是主角還是配角？

- AI 就是整個應用（衛教問答、個案評估助理）→ Dify
- AI 只是流程中的一步（收到表單 → AI 摘要 → 存檔 → 通知）→ n8n
- 這個問題想清楚，選型就決定了八成

為什麼課程主軸選 Dify

- 本課程的重心是「LLM 應用」而非「系統整合」
- RAG 知識庫在 Dify 是內建功能；在 n8n 要自己組向量庫、切塊、召回
- 但你應該知道 n8n 的存在——出社會後遇到整合需求，它才是對的工具

工具全景：兩個軸看清楚各自的位置

橫軸＝連得到多少外部服務　｜　縱軸＝ RAG 、 Prompt 管理、 Agent 編排的完整度



★ 實務上最強的組合： n8n 負責整合與觸發　＋　Dify 負責 AI 推理。

沒有「最好的工具」，只有「最貼近這個需求的工具」——看清楚需求的形狀比較重要。

課程涵蓋： M8 教 Dify （縱軸）、本節認識 n8n （橫軸）、 M10 教程式碼路線（跳出這張圖）。

橫軸　整合廣度

能連到多少外部服務。 Zapier ／ Make ／ n8n 在這一軸上遙遙領先。

縱軸　AI 深度

RAG 、 Prompt 管理、 Agent 編排的完整度。 Dify 在這一軸最強。

★ 右上角是空的

沒有工具同時做到兩者最好。所以實務上的解法是「組合」，
不是「找一個全能的」。

課程涵蓋範圍

M8 教 Dify （縱軸）、本節認識 n8n （橫軸）、
M10 教程式碼路線（跳出這張圖）。

★ 原生圖形，可直接在 PowerPoint 編輯

六個平台的定位對照

平台	出身	最強的地方	最弱的地方	什麼時候選它
Dify	LLM 應用平台	RAG 知識庫、 Prompt 管理、 Agent 編排	外部服務要自己寫 HTTP 節點	AI 就是整個應用
n8n	工作流自動化	400+ 現成節點、排程與 webhook 、自架簡單	RAG 要自己組， AI 部分較陽春	AI 只是流程中一步
Flowise	LangChain 視覺化	貼近 LangChain 生態、彈性高	介面較陽春，偏開發者	已在用 LangChain 想視覺化
Coze ／扣子	C 端 Bot 平台	模板與外掛多、上手極快	客製與資料落地受限	做面向大眾的聊天機器人
Make ／ Zapier	SaaS 整合	整合數量最多、幾乎不用學	AI 能力最淺、自架不可	純串接、 AI 只做一句摘要
LangGraph 等框架	程式碼框架	邏輯完全客製、可測試可版控	要會寫程式	邏輯複雜到平台塞不下

★ 沒有「最好的工具」，只有「最貼近這個需求的工具」。看清楚需求的形狀比較重要。

n8n 的典型流程長什麼樣

一條真實的 n8n 流程：觸發 → 取資料 → AI 摘要 → 條件判斷 → 通知



注意這條流程的重心在哪裡

- 六個節點裡只有一個是 AI —— 其餘都在做「連接外部服務」與「條件判斷」
- n8n 的價值是那 400+ 個現成節點：Google Sheets、Slack、Gmail、資料庫、HTTP 都是拉一下就好
- 若用 Dify：讀試算表、發 Slack 都得自己寫 HTTP 節點（1.10 起才有排程觸發）
- 反過來也一樣：要在 n8n 裡做 RAG，得自己接向量資料庫、自己處理切塊與召回 —— 同樣事倍功半

★ 判準：AI 只是流程中的「一個步驟」→ 用 n8n；AI 本身就是「整個應用」→ 用 Dify。

這個問題想清楚，選型就決定了八成。

本課程重心是 LLM 應用而非系統整合；但你該知道 n8n 的存在——出社會後遇到整合需求，它才是對的工具。

六個節點只有一個是 AI

這就是 n8n 的重心所在：其餘五個都在做「連接」與「判斷」。

拉一下就通的節點

Google Sheets、Slack、Gmail、資料庫、HTTP、排程——
都是現成節點，不用寫程式。

若用 Dify 做這個場景

Trigger、讀試算表、發 Slack 全都得自己寫 HTTP 節點，
Dify 1.10 起雖有排程／webhook 觸發，外部串接仍不如 n8n。

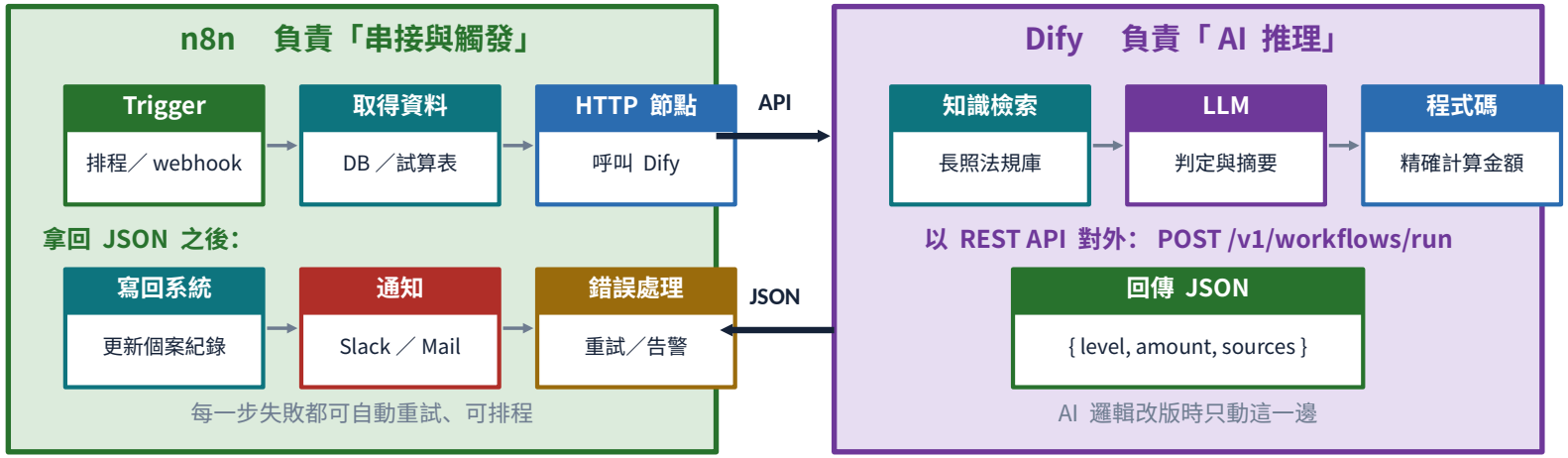
反過來也一樣

若要在 n8n 裡做 RAG，得自己接向量資料庫、

自己處理切塊與召回——同樣事倍功半。

★ 原生圖形，可直接在 PowerPoint 編輯

實務上最常見的正式架構： n8n ＋ Dify



為什麼這是實務上最常見的正式架構

- 各自做擅長的事：整合、排程與重試留給 n8n； RAG 、 Prompt 管理與精確計算留給 Dify
- AI 邏輯改版時只動 Dify ，不必碰整條自動化流程；自動化流程改了也不用重測 Prompt
- 兩邊各自可獨立測試——出錯時先分清楚是「接線壞了」還是「推理錯了」

★ 安全提醒： Dify 的 API 金鑰存在 n8n 的 Credentials ，不要寫死在流程節點裡。

匯出流程檔給別人、或上傳到 GitHub 之前，先確認金鑰沒有被一起帶出去。

分工

n8n 負責觸發、取資料、寫回、通知、重試；
Dify 負責知識檢索、 LLM 推理、精確計算。

接法

n8n 用 HTTP Request 節點呼叫 Dify 的 REST API （Workflow： /v1/workflows/run ；
Chatflow： /v1/chat-messages ），拿回 JSON 。

好處

AI 邏輯改版只動 Dify ，自動化流程不用碰；反之亦然。
兩邊各自可獨立測試。

★ 安全提醒

Dify 的 API 金鑰存在 n8n 的 Credentials ，不要寫死在流程節點裡，更不可匯出後上傳。

★ 原生圖形，可直接在 PowerPoint 編輯

同一個需求，Dify 與 n8n 各自要花多少力氣

需求	用 Dify	用 n8n	建議
衛教問答機器人 + 知識庫	內建 RAG，半天完成	要自己接向量庫，兩三天	★ Dify
每天 08:00 抓資料寄報表	1.10 起有排程觸發，但串接較費工	排程節點拉一下就好	★ n8n
表單提交後 AI 摘要並存檔	要自己寫 webhook 與寫回	Webhook + Sheets 節點現成	★ n8n
多輪對話的個案評估助理	Chatflow 直接支援對話歷史	要自己管 session 狀態	★ Dify
前面兩者兼具的完整系統	AI 強但整合弱	整合強但 AI 弱	★ 兩者串接

這張表可以直接當期末專題的選型參考。多數專題落在第一列與第四列，所以課程主軸放 Dify。

隨堂練習： AI 是主角還是配角？該用哪個工具

1 衛教問答機器人＋知識庫

Dify

內建 RAG，半天完成； n8n 要自己接向量庫，兩三天

2 每天 08:00 抓資料寄報表

n8n

排程節點拉一下就好； AI 只是其中一步

3 表單提交後 AI 摘要並存檔

n8n

Webhook ＋ Sheets 節點現成

4 多輪對話的個案評估助理

Dify

Chatflow 直接支援對話歷史； n8n 要自己管 session 狀態

5 每日讀試算表待處理個案 → 依長照法規判定 → 精確算金額 → 寫回並發 Slack

n8n + Dify

n8n 負責觸發、取資料、寫回、通知； Dify 負責檢索、推理與精確計算，中間用 HTTP 呼叫 Dify API 連起來

6 把院內衛教手冊做成可對話的查詢助理，並標註出處

Dify

AI 就是整個應用；知識庫、切塊、召回都是 Dify 內建

判準： AI 只是流程中的「一個步驟」→ n8n； AI 本身就是「整個應用」→ Dify；兩者都有 → n8n ＋ Dify。 Dify 的 API 金鑰存在 n8n 的 Credentials，不要寫死在流程節點裡。

若你想自己試 n8n： 三個入門提醒

環境比 Dify 輕

- 單一 Docker 容器、 2 GB RAM 即可； Dify 要 docker-compose 起 8 個服務、 4 GB 起跳
- 自架指令： docker run -it --rm -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n
- 有雲端版可直接註冊試用，不想自架也行

先看懂「節點 + 憑證」兩件事

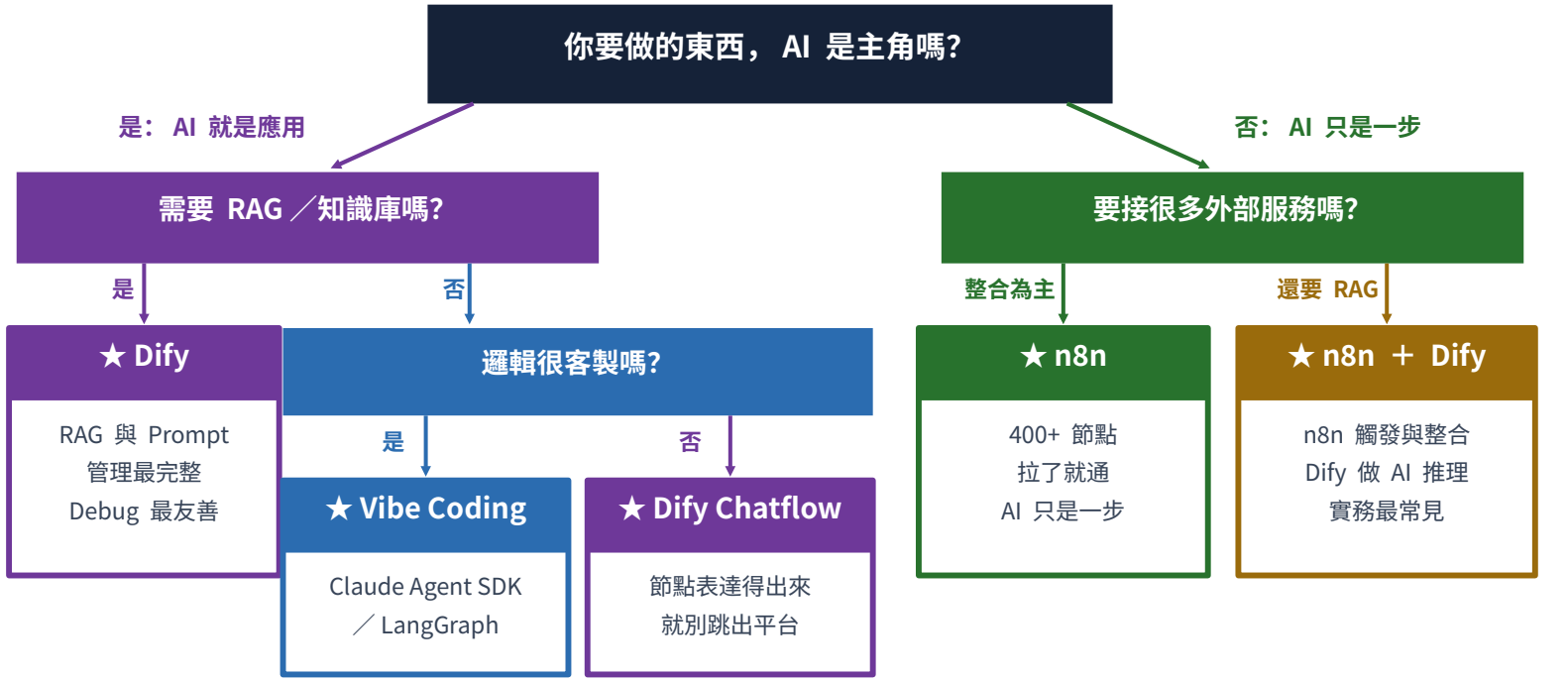
- 節點＝一個動作（讀試算表、發訊息）；憑證＝存放各服務的授權
- ★ 憑證絕不寫在節點參數裡，一律放 Credentials —— 匯出流程時才不會外洩
- n8n 也有 AI Agent 節點（基於 LangChain ），可掛工具與記憶

課程定位

- n8n 不列入必修，也不列入期末專題評分項目
- 屬於【自學】層級：想做整合類專題的同學可自行探索
- 另有補充教材《 n8n 入門與 AI Agent 實作》（含兩個完整範例）供自學

※ 本節內容為 2026 年 7 月查證之工具現況；此領域變動快，實際以官方文件為準。

選型決策樹：這個需求該用哪個工具



★ 給期末專題的建議：一律先用 Dify 把流程走通。

Dify 的 Debug 與 Tracing 對初學者最友善，出錯時看得見原因——這比「工具比較強」重要得多。

什麼時候該換路線： Dify 的節點真的表達不出你的邏輯時，才進 M10 的程式碼路線。

怎麼用這張圖

從最上面那個問題開始往下走，兩三步就會停在一個建議上。

第一個問題最關鍵

「AI 是主角嗎」決定了左右兩邊，也決定了八成的答案。

給期末專題的建議

★ 一律先用 Dify 把流程走通。 Dify 的 Debug 與 Tracing
對初學者最友善，出錯時看得見原因。

什麼時候該換路線

Dify 的節點真的表達不出你的邏輯時——這時進 M10 的程式碼路線。

★ 原生圖形，可直接在 PowerPoint 編輯

本模組重點回顧

1. Dify 是組裝平台，不是模型——它不會讓爛 Prompt 變好。
2. System Prompt 四段式：角色、任務步驟、限制邊界、輸出格式。缺一就會漂移。
3. Prompt 對「結構」約束力強，對「數量」與「數值」很弱——後者要靠流程節點與工具。
4. RAG 的品質瓶頸幾乎都在檢索，不在生成。 LLM 只看得到被召回的那幾段。
5. 數字一律查表或用工具算，不可由 LLM 生成——這是本模組最重要的一條規則。
6. 能用 Workflow 解決的就不要用 Agent；醫療場域一定要有人在迴圈裡。

下一步　第 16 週　期末發表：Dify Agent 專題 Demo　＋　M9 醫療 AI 的責任邊界